

3.3 THE SECOND ITERATION – FIRST PRACTICAL IMPLEMENTATION

The second iteration marks the change in the way the test jig framework hardware is organised. From this revision, for each test jig stack there are two (or more) of the generic test jig boards that will perform all of the testing on the UUT. The number of inputs and outputs required will determine the number of boards required. This iteration also marks the first attempt at implementation of the test jig framework to create a test jig for a guinea pig board (the MCU V3 PCB). This guinea pig board will continue to be used in future iterations to ensure that the jig creation process is as simple as possible. Since this was the first board explored with the framework, two new tests that were not anticipated in the original proposal were added, namely the ability to measure light and the ability to test for short and open circuits on the unit under test. In order to tell the test jig stacks apart, select lines were added, that allow each stack of base, stopper, guide and outline boards to have a unique id. The iCoupler isolated RS232 drivers have also been added to isolate the test voltages from the PC through the single isolation point. All of the I/O between the generic test boards and the test jig stack is now handled through 2 (or more) 40-pin ribbon cable connectors. The way test points are handled has been changed significantly by moving them from being mounted on the bottom base board, to the stopper board. By moving these test pins, the mechanical strain is removed from the base board where all of the electronic components are mounted for the test jig stack, and places all of the strain on the stopper board. The connections between the test pins and the base board are done by using solder type test pin holders, attaching wire leads to the test pins and then soldering these wire leads to the base board. Any strain now experienced due to the downwards force from the unit under test will warp the stopper board (which has no soldered components on it), but leave the base board undamaged. The impedance test capabilities of the test jig stack are achieved by placing a low voltage source in place of mains source to power the unit under test and then using the voltage drop across a series resistor to determine how much current is flowing through the unit under test. The optical measurement is achieved by using a light dependent resistor and a Wheatstone bridge measurement circuit to determine if light is present or not. Since the base design of the the hardware for the generic test jig board is similar between iteration 2 and iteration 5 of the design, the hardware will be discussed in iteration 5 in detail.

3.3.1 Iteration 2 Characteristics

Board	Rev
Generic Test Jig	B
MCU bed of nails base	A
MCU bed of nails stopper	A
MCU bed of nails guide	A
MCU bed of nails outline	A

Table 4: Hardware designs associated with iteration 2

- RS232 communication between PC and test jig
- operator interfaces to the jig using the PC
- PC to give feedback to the operator
- test results are stored on the PC
- Debug communication to UUT is handled by PC
- Test application determines the tests to run based on the bed of nails attached to it
 - automatic configuration for all types of jig in a single jar
- Single points of isolation – RS232 isolated by iCoupler device
- Measurements float on the same ground potential(s) as the UUT
- fixed interface connector to base of bed of nails, but more expandable as more connectors can be added for more generic test jigs beyond the standard 2
- fixed number of inputs/ outputs per generic test board – scalability and configuration achieved by adding more boards if required beyond the standard 2
- single processor on the generic test board and measurements done using the processor's on board peripherals
- protection on ADC and logic inputs achieved with series resistors and zener diodes.
- Additional measurement options are explored – LDR light measurement to measure led light added.
- Provision made for impedance test functionality.
- Select lines added to allow the test jigs to have ids.

The Full Schematics and gerber files for all of the boards in Iteration 2 are all available in Appendix C.

3.3.2 Test Jig Creation Process

The test jig creation process is different from the process outlined in the prototype in that not only has the hardware stack changed, but the way the tests are coded has also been changed. The process for creating the software on the jig in the second iteration is shown in the following diagram.

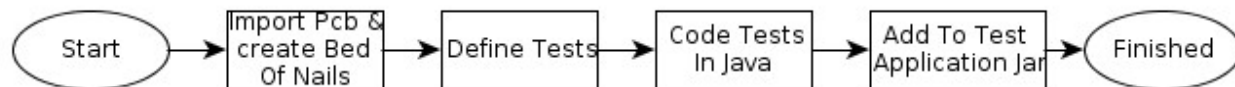


Illustration 22: Test Jig Creation Process

The jig creation process is now slightly simpler than previously envisioned. In this iteration, the process has been refined in that proposed templates for the entire stack have been put forward, and a number of standard tests have been developed in the Java application framework, which are supported by the firmware on the generic test boards. The test jig stack creation process has been changed to be as followed:

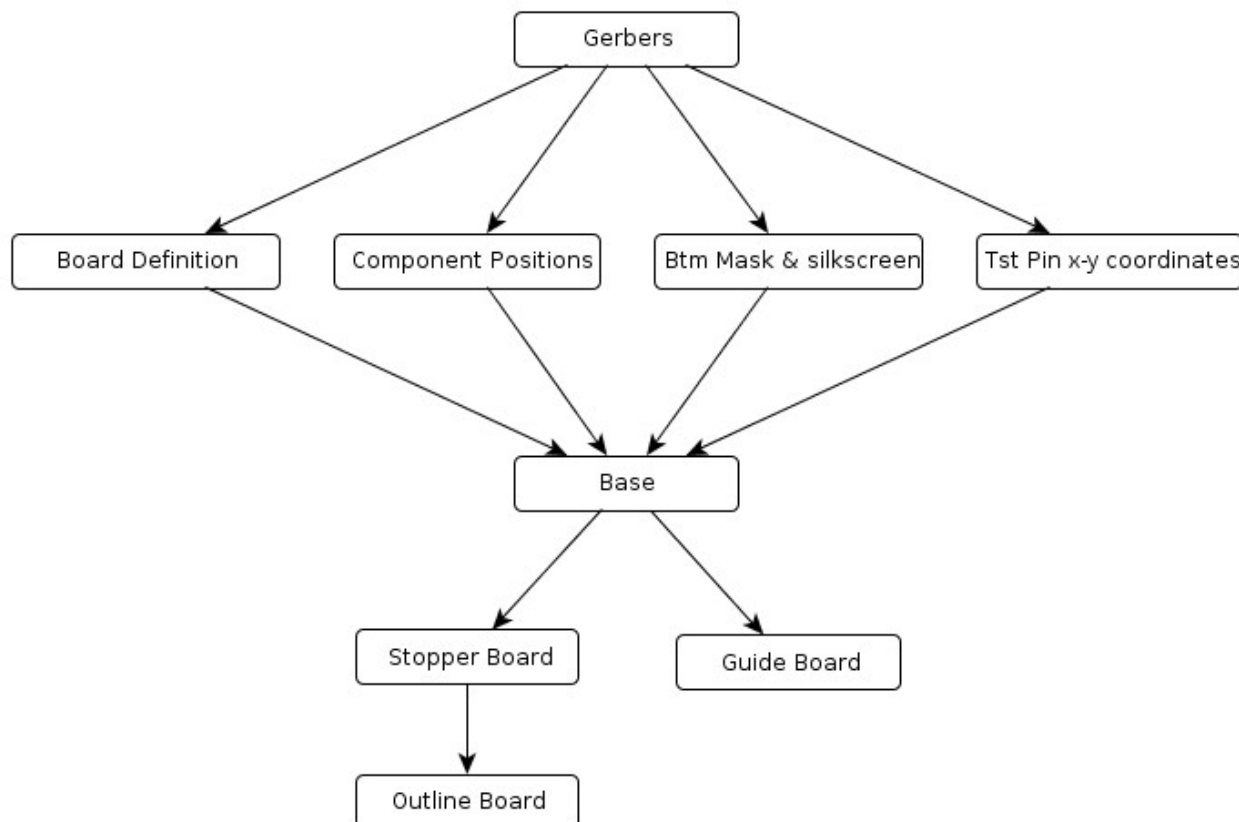


Illustration 23: Creating The Test Jig Stack In Iteration 2

- **Base Board**

The base board is created by importing the unit under test's gerber files into the template and placing it in the center of the board. From this reference point, the outline of the unit under test's gerbers needs to be traced onto the base board, and test points placed for every point to be measured. These test points won't have the test points soldered to them, only jumper wires coming from the board above it (the stopper board). The base board is also used to do all of the signal conditioning for the test points.

- **Stopper Board**

The Stopper board is where the test pins are attached to. From the bottom of the test pins, jumper wires are soldered, which then drop down and are soldered to the base board in the test point holes created earlier. To build the stopper board, the base board is just copied, all of the components removed, and the test pin holes are changed to 68 mil holes to hold the test pins. For the LDRs, jumpers need to be dropped down and connected to a second set of jumper holes which will house the LDRs on the stopper board.

- **Guide Board**

The Guide board is what the unit under test rests on while allowing the test points and all other through hole components to push through it. It is generated by using the imported gerbers used in creating the Base board, and placing appropriately sized holes for each of the test pins and through hole component leads.

- **Outline Board**

The outline board is created using the stopper board. A cut out is drawn around the silkscreen outline of the unit under test board, approximately 0.5mm bigger in each direction so that the board can drop in easily.

This design was first mocked up mechanically using cardboard and PCB standoffs to attempt to ensure it overcame the problems experienced in the prototype. The board used for the mock up was the generic test jig board itself.



Illustration 24: Test Jig Stack Mock Up

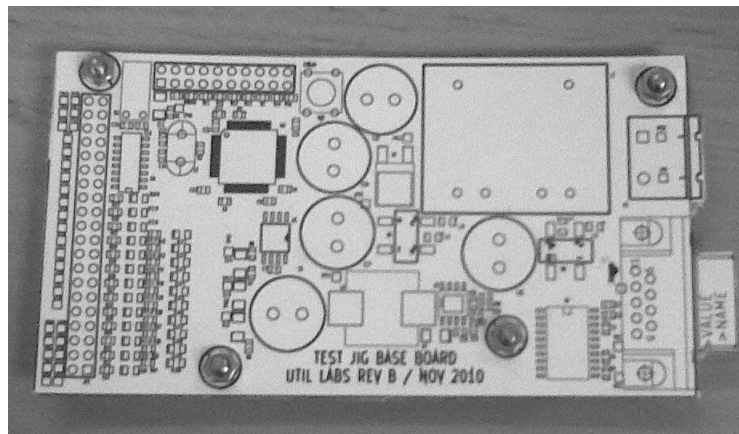


Illustration 25: Test Jig Generic Base Mock Up

3.3.3 Design Review – Changes Made To Iteration 2 To Create Iteration 3

The Changes between the prototype and first implementation were very large, and as such some problems crept in.

- **Board Layout**

Problem: There were some problems with the board layout:

- on the power supply, the 12v regulator footprint (use the standard through hole version) was incorrectly specified as the D2PAK instead of the through hole TO220 in stock.
- the iCoupler doesn't work – it needs either an additional 5V power supply on the input side, or a 3.3V supply on the output side [24, page 3 table 1]. By just running it on the 3.3V supply on the VCC rail, the iCoupler is run with the DC-DC converter on the isolated side disabled.
- The transformer used is a little under powered, and runs fairly hot. Use a larger transformer. There is also no use in running two secondary windings on the transformer.
- There is a problem in the kicad library, in that the db9 connector doesn't have the solder pads masked out. fix solder mask problem in the library.
- There is a problem in the kicad library, in that the pin arrays (header pins) have pin 1 of the connector smaller than the other pins. Fix footprint.
- The 0603 resistors used on the board are difficult to solder by hand, and due to the expected number of boards that will be made, it is likely that the boards will be soldered by hand. Change the 0603 resistors to 0805.

- **Test Pin Connection Between Base And Stopper**

Problem: The initial implementation of the test pin connection between the base board and stopper board proved to be very very difficult to implement mechanically. Although good electrical contact was made between the UUT and the test pins, the physical act of wiring up the test pin holders to the base test pin solder holes made the design inviable, as to allow the base and stopper boards to be split, the wires needed to be <15cm, and this meant a huge amount of wire in between the base and stopper boards when the boards were bolted together. The same action that supplies the protection to the wiring (the fact that it is springy, and doesn't rigidly press against the base board), also proved problematic, as the test pin holders

were now being pushed up.

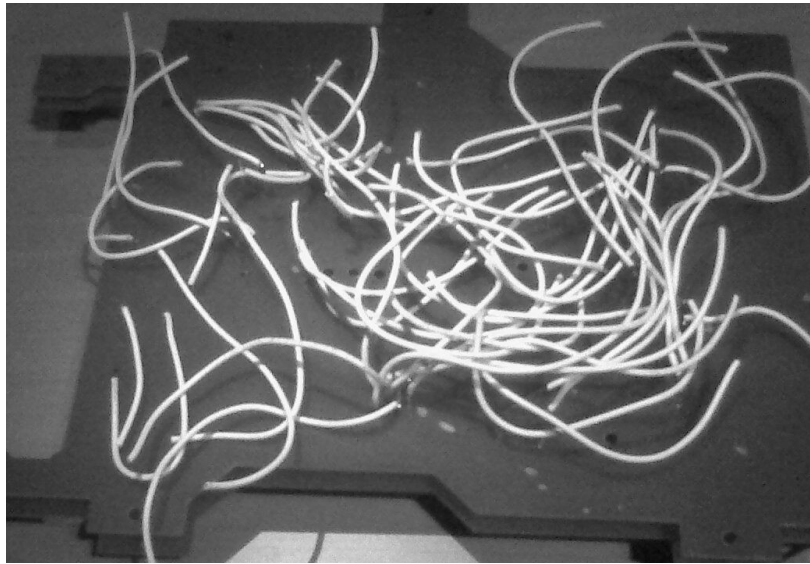


Illustration 26: Iteration 2 Implementation Bottom

Solution: The solution to this problem was twofold – solder the test pin holders onto the stopper board, and instead of wiring from the back of the test pin holder to the base board, use an additional set of ribbon cable to wire between the base board and stopper board. Since the test pin holders would now be soldered onto the stopper board, a different solution would be required that would stop the board from flexing and breaking the solder connections. This is achieved by using 2 stopper boards, held apart by a number of 5mm spacers. The two boards have significantly more physical strength, and as such prove to not flex under the pressure of all of the test points.

- **Non-Linear Force**

Problem: The initial idea was to have the test points push through the guide layer, and use bright bar (or similar) guides to allow the operator to push the board down with a clamp onto them. This method however proved very difficult, as it was not possible to ensure that the operator presses down 90 degrees onto the board, and any bending of the test pins cause them to press against the guide layer, and not be depressed properly and in some cases be damaged due to the physical pressure being more on some pins than others.

Solution: The solution to this, was to rethink the way the pins connect to the UUT, by instead of making the operator apply downward force onto the pins, allow the operator to gently place the board onto the guide layer without the test points

present, using the outline layer as a guide, clamp it down and then pressing the test pins up into the testing position using a lifting plate arrangement. This single axis of movement for the test pins meant there is no chance for the pins to be bent or the operator to apply the pins incorrectly, as it would be applied with the pull of a lever.

- **Jtagconnection**

Problem: The high speed lines for the JTAG programming need to have shorter track lengths to ensure proper data transfer from the JTAG to the UUT.

Solution: The arrangement of having all of the connectors on the base board is unnecessary, the debug comms and the JTAG connectors no longer go to the base board and as such can be routed on the stopper board to keep these high speed lines much shorter.

3.4 THE THIRD ITERATION

The third iteration marks the change in the responsibilities of the boards in the test jig stack. The following responsibilities are assigned to each board :

- **Base :** The base board is responsible with the stopper board for the signal conditioning of the signals to and from the test pins. The resistor dividers, signal relays and low voltage sources are placed here.
- **Stopper:** The Stopper board is where the test pins are attached to. There is also some signal conditioning done on the stopper board in the case of the LDRs that sense the light signals from the board and their corresponding Wheatstone bridge circuit. There are two stopper boards placed one on top of the other with 5mm spacers in between to ensure mechanical stability of the board.
- **Guide:** The unit under test rests on the guide board during the test sequence once it has been clamped down. As such, the guide board has holes cut for all of the test pins to pass through it when the lifting plate is actuated, as well as having holes cut out for all of the components mounted on the bottom board and any through hole component pins to ensure that the unit under test rests flush on it.
- **Outline:** The outline board is used to position the unit under test on the guide board before it gets clamped down mechanically for the test.

The connections between the base board and stopper board are now done with the same type of ribbon cable as is used to connect between the generic test jig boards and the base board. This iteration was the first iteration in which the iCouplers were shown to be an effective isolation point, in that no damage was done to the PC that housed the testing software even when mains voltage was present on the test pins on the test jig stack.

3.4.1 Iteration 3 Characteristics

Board	Rev
Generic Test Jig	C
MCU bed of nails base	B
MCU bed of nails stopper	B
MCU bed of nails guide	B
MCU bed of nails outline	B

Table 5: Hardware designs associated with iteration 3

- RS232 communication between PC and test jig
- operator interfaces to the jig using the PC
- PC to give feedback to the operator
- test results are stored on the PC
- Debug communication to UUT is handled by PC
- Test application determines the tests to run based on the bed of nails attached to it
 - automatic configuration for all types of jig in a single jar
- Single points of isolation – RS232 isolated by iCoupler device
- Measurements float on the same ground potential(s) as the UUT
- fixed interface connector to base of bed of nails, but more expandable as more connectors can be added for more generic test jigs beyond the standard 2
- fixed number of inputs/ outputs per generic test board – scalability and configuration achieved by adding more boards if required beyond the standard 2
- single processor on the generic test board and measurements done using the processor's on board peripherals
- protection on ADC and logic inputs achieved with series resistors and zener diodes.
- LDR light measurement.

The full schematics and gerbers for all files in Iteration 3 are available in Appendix D.

3.4.2 Test Jig Creation Process

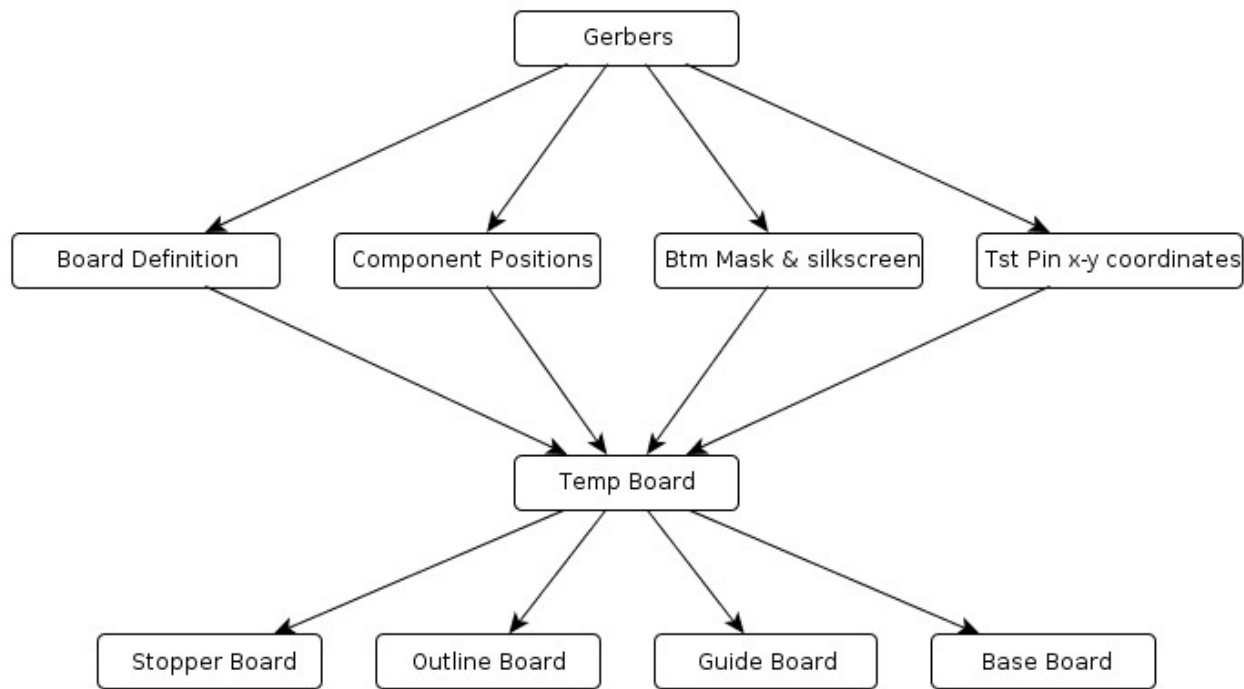


Illustration 28: Test Jig Stack Creation In Iteration 3

In Iteration 3, the use of the template is explored further and updated to take into account the changes in the responsibilities of the different boards in the test jig stack. The gerber files for the UUT are first pulled into a temporary board used for creating the other boards. The board outline of the UUT, test point positions, all through hole components and components mounted on the bottom of the UUT need to be taken into account. This information is placed in the middle of the Temp board, and then saved as temp-
boardname.brd, where *boardname* is some identifier for the UUT. From the temp-
boardname.brd file, all the other boards can be created. The schematics for the stopper and base board need to take into account all of the mounting holes on temp-
boardname.brd, as well as the pre-placed connectors P1, P2, P4 and P5. The stopper board schematic also needs to take into account the select resistors R1-R10. In the test jig stack, 2 stopper boards are used with a number of 5mm spacers in between to give the boards mechanical rigidity. Place sufficient screw holes in temp-*boardname*.brd to keep the boards rigid when they are bolted together. As a rough guide, all of the test points need to be at least 1 inch away from a stabilising screw.

- **Base Board**

The base board is created by removing from temp-*boardname*.brd, all of the components that were imported, as well as the select resistors R1-R10. This leaves only the inter-board connectors P1,P2,P4 and P5, the outline for the UUT, the test points and the stabilising screw holes (not all of these will have pillars down to them, but it is good to keep tracks away from those points if pillars are required later for stability on the stopper board). The outline on the board is now changed from the “board definition layer” to the “top silkscreen” layer so it serves only as a visual indication on the base board as to where the UUT is positioned. From this point the schematic is created (with P1,P2,P4 and P5 present) routing all of the lines that need to go to test points on the stopper board using the mapping for P1 and P2. The mappings for P1 and P2 are as follows:

Pin	P1 (BoardA)	P2(BoardB)	Pin	P1(BoardA)	P2(BoardB)
1	Select1	Select1	21	Analog5	Analog5
2	Select0	Select0	22	GNDA	GNDB
3	Select3	Select3	23	Analog4	Analog4
4	Select2	Select2	24	GNDA	GNDB
5	Input0	Input0	25	Analog1	Analog1
6	Select4	Select4	26	GNDA	GNDB
7	Input1	Input1	27	Output0	Output0
8	Input2	Input2	28	GNDA	GNDB
9	Input3	Input3	29	Output1	Output1
10	Input4	Input4	30	GNDA	GNDB
11	Input5	Input5	31	Output2	Output2
12	Input5	Input5	32	GNDA	GNDB
13	Input6	Input6	33	Output3	Output3
14	Input7	Input7	34	3.3VA	3.3VB
15	Analog9	Analog9	35	Output4	Output4
16	Input8	Input8	36	3.3VA	3.3VB
17	Analog15	Analog15	37	Output5	Output5
18	Analog14	Analog14	38	12VA	12VB
19	Analog7	Analog7	39	Output6	Output6
20	Analog6	Analog6	40	12VA	12VB

Table 6: P1 and P2 mappings on temp-*boardname*.brd

The Analog lines are marked for the channel number on the STM32F103 processor's ADC peripheral. Once all of the signal conditioning has been added to the schematic, the netlist can be imported into PCBnew, and the board can be laid out taking into account the positions of the test pins (to not place components over them). Once the board has been laid out, the test points can be removed. The board designator on the PCB silkscreen should be changed from "Temp" to an appropriate board designator like:

BOARD NAME BASE

COMPANY NAME REV *REVISION* / *MONTH* *YEAR*

- **Stopper Board**

The Stopper board is created by removing from temp-*boardname*.brd, all of the components that were imported (assuming none of the components on the bottom of the board are higher than 14mm), and leaving only the UUT board outline on the "top silkscreen" layer, the test points, the mechanical stabilising screw holes and the connectors P4 and P5. Using the mappings on P4 and P5 that were created during the Base board creation, create the schematic for the stopper board taking into account P4 and P5 as well as the test points. Once the schematic is created, feed the netlist into PCBnew and lay out all of the required tracks. The board designator should be updated as above using the following format:

BOARD NAME STOPPER

COMPANY NAME REV *REVISION* / *MONTH* *YEAR* .

- **Guide Board**

The Guide board is created from temp-*boardname*.brd by removing all of the added connectors, leaving the outline of the UUT on the "top silkscreen" layer, and then using the "bottom silkscreen" layer of the UUT as well as the test point positions and the pin positions of all through hole components to create a board that the UUT can lie flat on during the test, allowing all of the test pins, components and component leads to pass through it without obstruction. In positions where there is a large density of holes that need to be made, a cut-out can instead be made that will allow groups of pins/components/leads to pass through.

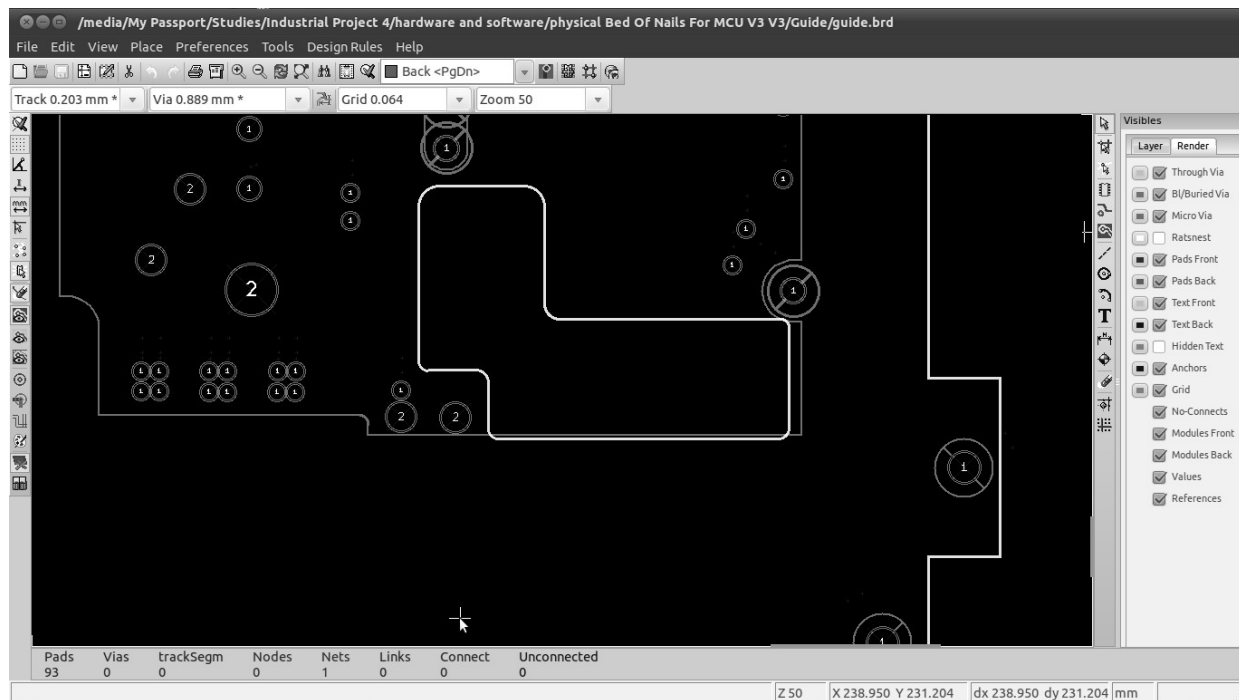


Illustration 29: Guide board showing a cut-out made for a dense number of test points

- **Outline Board**

The Outline board is created from temp-*boardname*.brd by removing everything but the board definition of the UUT. An outline needs to be drawn in the same around this about 0.5mm away from the original UUT outline. This allows the UUT to rest on the Guide board, but not move forward and backward during the test.

3.4.3 Design Review – Changes Made To Iteration 3 To Create Iteration 4

The Third iteration moved much further towards the final design of the generic test jig system, and allowed the first successful bench tests to be done. The first meeting with the Mechanical department was held, and the outcome was to use the templates of the boards as they stand at the moment and do a first iteration of the physical test jig hardware, including the covering plate for the UUT, as well as the lifting plate arrangement. For more information on the mechanical hardware, see section 3.7. The following 2 iterations (4 and 5) were used to optimise the design, and to fix any latent issues in the generic test jig board and test jig stack. The issues to address in the following iteration are as follows:

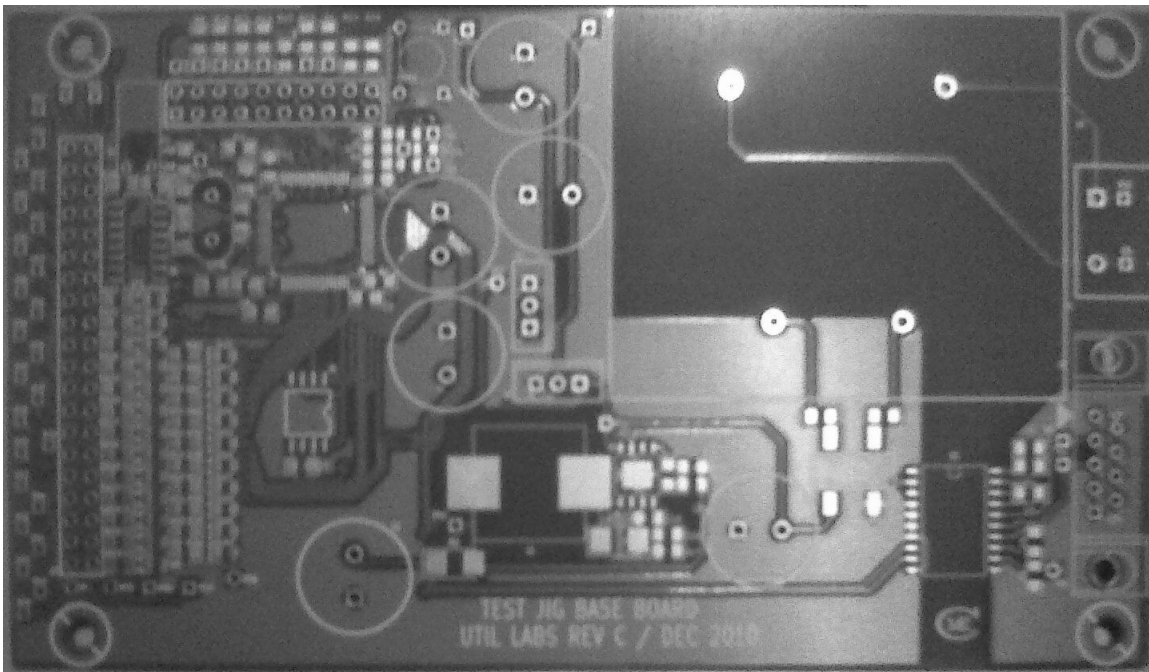


Illustration 30: Generic Test Jig Rev C before population

- **Future expansion limitations due to select lines**

Problem: In this iteration the number of boards that the generic test jig can handle is limited to 32 since both of the select line sets reflect the same value allowing you to identify both of the boards as being connected to a jig of ID “00001”. The problem with this arrangement, is that it doesn't allow you to identify which of the two individual generic test jig board is which (as they play different roles) and the 5 bit select lines will only allow a maximum of 32 different test jigs.

Solution: By combining the two values, and making the ID a 10 bit value with the least significant 5 bits connected to one generic test jig board and most significant 5 bits connected to the other one, there is the opportunity to make almost 1000 jigs (using the full 1024 values wouldn't work, as there are a number of times when jigs would have 2x 5 bit ids that are the same in the 10 bit value, which would lose the advantage of being able to tell the jigs apart). In order to tell apart which generic jig board is which, the “left” jig connecting to the test jig stack will always be connected to the least significant 5 bits.

- **Impedance Test circuit doesn't function correctly**

Problem: The impedance test circuit doesn't function correctly, and always reads the same value.

Solution: Re-bias the circuit

- **Confusing naming convention**

Problem: in the process of mapping each test pin to an input/output on the generic test jig boards, the naming is confusing to follow.

Solution: fix the logical mapping and wiring of the board (ensure test pins got to the correct positions), by streamlining the naming convention of the inputs/ outputs and specifically the analog measurement pins.

- **Simplify wiring**

Problem: Wiring the test jig stack is overly complex as no provision is made for a standard way to connect the peripheral hardware in the test jig box to the generic test jig controller.

Solution: Add connectors for all of the test jig hardware (Interlock, Counter, Indicator Leds RUNNING, PASS, FAIL) to the front of the bed of nails stack allowing all the connectors to be accessible from 1 side. The only connector to keep separate would be the mains input to the impedance measurement circuit, as this will not be fitted for all test jigs, and also simplifies the creepage and clearance requirements. (it is moved slightly though to make the bed of nails compatible with the generic mechanical jig V1.)

- **Zener Diodes leakage affect measurements**

Problem: The zener diodes were added between the prototype and first revision of the test jig, and were to protect the CMOS chip inputs and ADC inputs on used on the prototype. The problem with the zener diodes are their high leakage current (depending on the voltage applied, as high as 200uA) found in experimentation showed to create a voltage drop of up to 0.2V in the series resistor, which badly affected the analog measurements.

Solution: Remove the zener diodes. The stm32F103 has internal protection diodes, and with the addition of the series resistor, these diodes offer sufficient protection to over voltage situations. The zeners need not be removed from the PCB, just not populated on the BOM.

- **Excessive heat on high load**

Problem: When running for a while, 12V, 5V regulators as well as the iCoupler tend to run really hot due to the amount of current consumed by the iCoupled (145mA). This problem is made significantly worse when large numbers of relays are switched.

Solution: Add additional heat sinking to the iCoupler to handle excess heat, and add the capabilities for an external power supply if required (the regulator alone is sufficient for the running of the iCoupler and a few relays, but when switching a large number of times this will become a problem).

3.5 THE FOURTH ITERATION

The 4th iteration was used as a mostly optimization exercise, in that the 3rd iteration was already working but some small problems needed to be fixed with reference to heat and physical wiring. In order to fix the heat generated on the PCB and the strain placed on the transformer, the generic test jig board was changed to allow for either using the on board power supply or an external power supply and by adding significantly more heat syncing on the board for the iCoupler chip. To ensure that only the external power supply or the on-board power supply are used, the components are placed in such a way as to make it impossible for both options to be fitted (this part of the design is covered in detail in the discussion of the Fifth iteration in Section 3.6). In order to simplify the wiring of the jig, connectors to the mechanical system were chosen such that it is difficult to wire the jig incorrectly. There is some responsibility added to the base board to carry the connector to wire the mechanical interlock and counter, and for the stopper board to carry the connector for the wiring of the indicator LEDs. The connectors used to interface between the test jig stack and the mechanical jig are as follows:

- 2-pin: a two pin socket 2EDGRC-7.62-02P is used to allow for the adding of mains to the test jig stack_[26]. The mating plug that goes into the socket is 2EDGK-7.62-02P_[25].
- 3-pin: the three pin socket 2EDGRC-7.62-03P is used to connect the mechanical counter and mechanical interlock. The mating plug for the socket is 2EDGK-7.62-03P.
- 4-pin: the four pin socket 2EDGRC-7.62-04P is used to connect the indicator lights that show the current testing state to the operator to the test jig stack. The mating plug for the socket is 2EDGK-7.62-04P.
- 40 pin headers: there are 2 x 40 pin ribbon connectors on the test jig base board that go to the generic test jig boards. The length of these ribbon (5cm) cables were chosen such that it is not possible to connect the generic test jig boards incorrectly to the test jig stack.
- 20 pin headers: the 20 pin header is for the JTAG

Using these standard connectors, the base and stopper templates were created to allow for rapid test jig creation, and the process for creating a test jig rapidly was tested out by

creating a test jig for another product. The rapid test jig creation process was a resounding success.

3.5.1 Iteration 4 Characteristics

Board	Rev
Generic Test Jig	D
MCU bed of nails base	C
MCU bed of nails stopper	C
MCU bed of nails guide	C
MCU bed of nails outline	C
Display bed of nails base	A
Display bed of nails stopper	A
Display bed of nails guide	A
Display bed of nails outline	A
Base Board Template	A
Stopper board Template	A

Characterised by

- RS232 communication between PC and test jig
- operator interfaces to the jig using the PC, and with 3 simple test status indicator LEDs.
- PC to give feedback to the operator
- test results are stored on the PC
- Debug communication to UUT is handled by PC
- Test application determines the tests to run based on the bed of nails attached to it
 - automatic configuration for all types of jig in a single jar
- Single points of isolation – RS232 isolated by iCoupler device
- Measurements float on the same ground potential(s) as the UUT
- fixed interface connector to base of bed of nails, but more expandable as more connectors can be added for more generic test jigs beyond the standard 2

- fixed number of inputs/ outputs per generic test board – scalability and configuration achieved by adding more boards if required beyond the standard 2
- All connectors to the bed of nails now accessible from one side to facilitate easier swapping of beds of nails.
- Physical interlock to stop the mechanical test jig cover from being opened while a test is in progress, and also to help ensuring that the correct label is placed on the correct device when testing in parallel with a single printer (scan to ensure the device and label serial numbers are the same)
- single processor on the generic test board and measurements done using the processor's on board peripherals
- protection on ADC and logic inputs achieved with series resistors and zener diodes.
- LDR light measurement to measure led light.
- impedance test functionality.
- Connectors for external power supplies.
- First rapid test jig created to prove that the project fulfills the initial requirements.
- First test jig stack to be fitted into the mechanical Jigs

The full schematics and gerbers for Iteration 4 are available in Appendix E.

3.5.2 Test Jig Creation Process

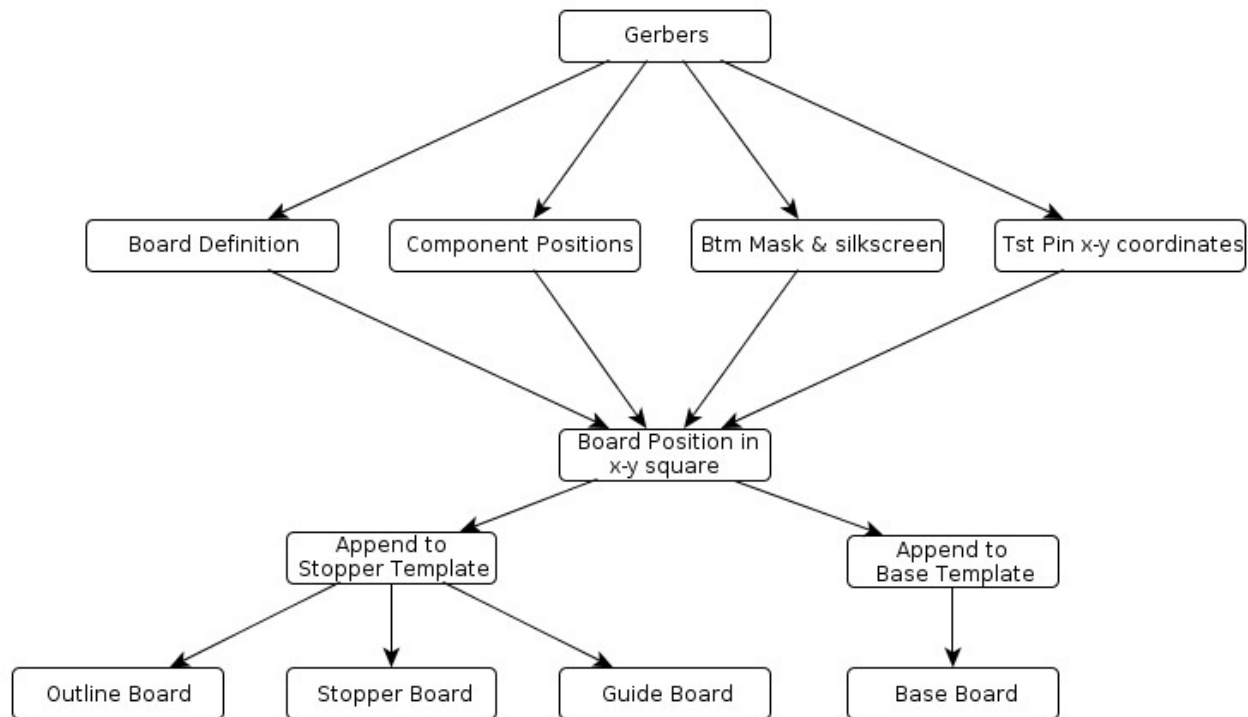


Illustration 31: Iteration 4 Test Jig Stack Creation

The creation of the test jig stack has also been optimised in Iteration 4. Two templates are now used to create the boards in the stack, and the UUT gerbers are “Appended” onto the templates one at a time. Append is a function available in Kicad, which takes one board as a brd file, and places it at the exact same x-y position on another board, and then creates from this an intermediate board called *boardB*-append.brd. To use this, an x-y area is defined, that will allow the UUT to sit in the middle of the test jig stack, and the gerbers are appended to the two templates. To create a .brd file from a set of gerbers, the GerbView utility is used from the Kicad suite. once the brd file is created from the gerbers, the board is moved to the designated x-y coordinates and then appended. The x-y coordinates are defined by a box with the following corners:

Corner	X position	Y position
Top Left	110mm	80mm
Top Right	230mm	80mm
Bottom Left	110mm	230mm
Bottom Right	230mm	230mm

Table 7: Maximum UUT Board Size For Iteration 4

As can be seen, this gives a maximum UUT board dimension of 120x150mm. Once the board has been appended, the boards in the stack are created as follows:

- **Base Board**

Using the Base-append.brd board created by appending the UUT board, the base board is created by removing everything but the board outline (which is converted to the top silkscreen layer) and the test point positions, so that the schematic can be created (using the base template schematic) that just routes the correct lines from the generic test jig boards to the stopper board where the test points are housed through some intermediate signal conditioning placed on the base board. Once the schematic is created, it can be pulled into the base-append.brd board and laid out taking into account the positions of the test pins. Once the board has been laid out, these test pin positions can be removed. The Base-append.brd board designators need to be updated to show that the board is now a base board for a specific UUT board.

- **Stopper Board**

The stopper board is created using the Stopper-append.brd created above. The outline of the UUT board is created as a silkscreen and the test pins are populated with test pins in the appropriate positions. Once the test pins have been placed on the board, the schematic can be created by taking the stopper schematic template and adding the test points to it. Any additional signal conditioning not done on the base board (like for example placing transducer hardware to read light as opposed to electrical signals) is now added to the schematic. To communicate with the UUT via a debug port, an ICoupler is added to ensure that the electrical isolation is maintained between the UUT and test PC. The ICoupler is pre placed on the Stopper-append.brd with an appropriate amount of heat syncing and 4 line stubs that are used to connect +5V,gnd,RX an TX to the ICoupler.

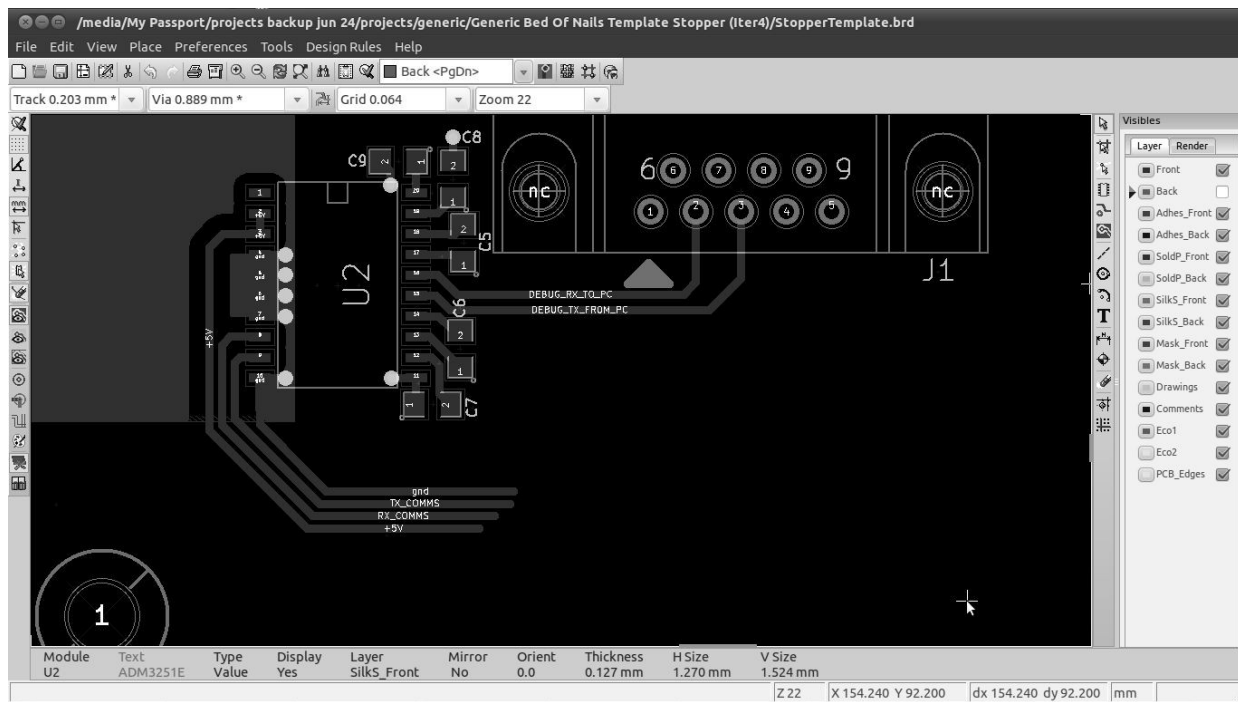
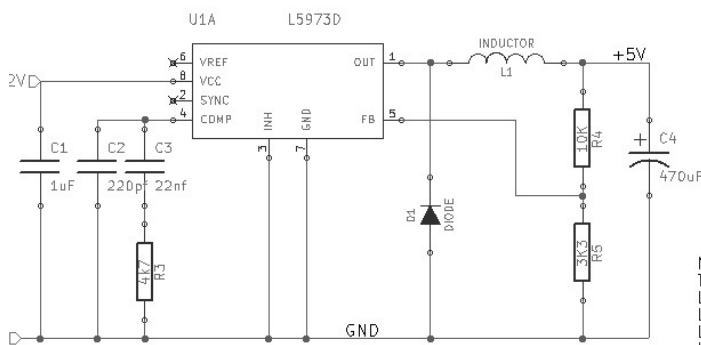


Illustration 32: Stubs On ICoupler On Stopper Template

To power the chip either a pre-laid schematic of a switch mode power supply can be used, that must then be laid out around the test pins, or +5V can be brought up from the base board. If the power supply is brought up from the base board, then the U1 circuit can be removed from the schematic.



Notes:
 The +5V power supply for the Icoupler can either come from the U1 circuit or be brought up from the base, in which case the U1 circuit can be removed.
 UUT_RX goes to the DEBUG_RX test point on the UUT
 UUT_TX goes to the DEBUG_TX test point on the UUT
 JTAG pins each go to the corresponding JTAG test points

Illustration 33: Optional Switch Mode Power Supply On Stopper Template

Once the schematic is created, the netlist can be pulled into the board and laid out. The Stopper-append.brd board designators need to be updated to show that the board is now a stopper board for a specific UUT board.

- **Guide Board**

The guide board is created using the Stopper-append.brd created above. The outline of the UUT board is created as a silkscreen, and then the bottom silkscreen of the UUT board is used to create cut outs for any components mounted on the bottom of the board, and the test points and through hole lead positions of other components on the top layer of the UUT are used to create holes for those leads to go through. All of the components on the Stopper append board are then removed. There is no schematic for the guide board and therefore no tracks, and no components are fitted onto it. When manufacturing the Guide board, only the top silkscreen layer need be sent along with the board outline layer so that the manufacturer knows not to add any copper. The Stopper-append.brd designators need to be updated to show this is now a Guide board for a specific UUT board.

- **Outline Board**

The outline board is also created from the Stopper-append.brd board created above. The only step required for making an outline board is to draw in the Board definition layer an outline around the UUT at a distance of 0.5mm away from it and then removing all of the components on the board. The board designators need to be updated to show this is a outline board for a specific UUT board. The outline board has no schematic or copper layers.

3.5.3 Design Review – Changes Made to Iteration 4 to Create Iteration 5

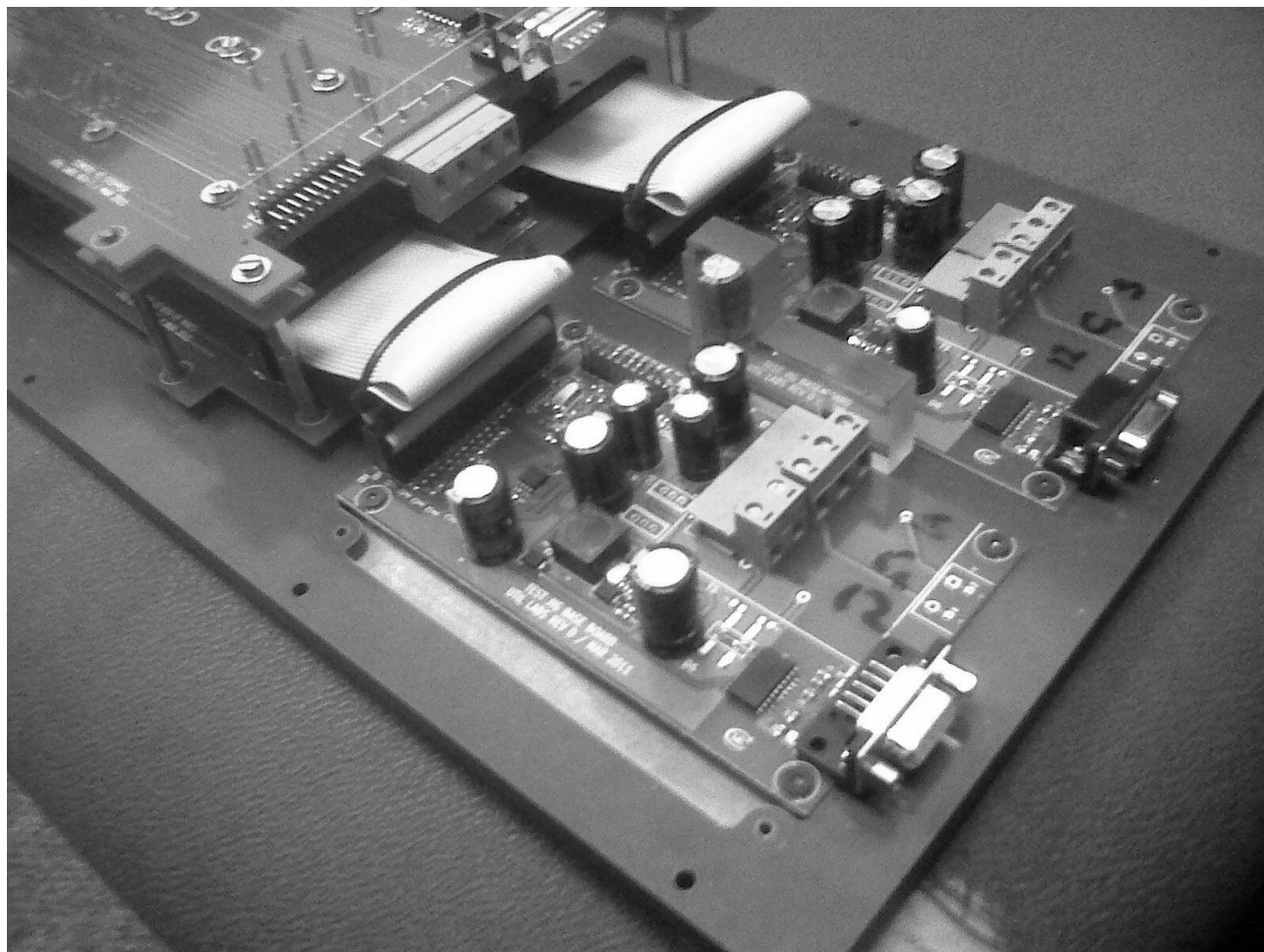


Illustration 34: Fitting Iteration 4 Boards On the Mechanical Base Plate

Version 4 worked well as a test jig stack on the bench, but in attempting to fit it into the 2nd version of the mechanical hardware, there were some problems in seating the stack in the jig. The first revision of the mechanical jig was created around the 3rd and 4th iteration, but a more generic lifting plate test jig frame was found, and with small modifications found to be a suitable solution for the generic test jig. Although the generic mechanical frame was a suitable solution, it was not 100% compatible with the 4th iteration and as such the following mechanical changes need to be made:

- All of the connectors on the stopper board are to be mounted on the bottom layer instead of on top of the board.
- The screws around the edge of the stopper assembly should be placed in set positions so that they can press into the holes provided on the mechanical faceplate.

- Shorter ribbon cables need to be used in the test jig stack to stop mechanical interference.
- The switching probes used to detect board presence are just too expensive, so an alternative solution to presence detection needs to be explored.

3.6 THE FIFTH ITERATION

The 5th iteration is the final iteration in terms of the initial project proposal. It marks the successful implementation of a test jig in a mechanical housing and successful testing of a board supplied from a manufacturer by a qualified operator. The templates and generic test jig boards were finalised into a release version and all documentation done for the creation of test jig stacks and the programming of test sequences. The flexibility of the generic test jig system was also shown, in that the board used as a guinea pig throughout the course of this project (MCU V3) changed during this iteration, and the changes were made in the test jig stack to accommodate both the original MCU V3 board as well as the MCU V3 (Rev D) without any problems. The only major design changes was to stop using switching probes to detect board presence due to the high cost of these pins and their respective holders and to instead opt for a simple micro switch, and the changing of the stopper assembly to now have a bottom board that accommodates the connectors mounted on the bottom of the stopper PCB (Backplane). The templating process was greatly optimised in this iteration, creating a positioner board that allows the UUT under test to be positioned for each board in the stack, and this even allows multiple people to work on a stack at once. Full isolation is also now a reality as all of the communication lines are now as a standard (due to the template) isolated using ICouplers, and the JTAG can be isolated using a JTAG isolation board from Segger (available from Digikey as part number 899-1003-ND)^[13]

3.6.1 Iteration 5 Characteristics

Board	Rev
Generic Test Jig	D
MCU Rev D bed of nails base	C
MCU Rev D bed of nails stopper	B
MCU Rev D bed of nails guide	A
MCU bed of nails outline	C
MCU Rev D backplane	A
Base Board Template	B
Stopper board Template	B
Backplane Template	B
Guide Template	B
Outline Template	B
Positioner	A

Table 8: Hardware designs associated with iteration 5

- RS232 communication between PC and test jig,UUT
- Full Isolation between test jig PC and UUT on all communication lines through ICoupler devices or opto couplers.
- operator interfaces to the jig using the PC, and with 3 simple test status indicator LEDs.
- PC to give feedback to the operator
- test results are stored on the PC
- Debug communication to UUT is handled by PC
- Test application determines the tests to run based on the bed of nails attached to it
 - automatic configuration for all types of jig in a single jar
- Measurements float on the same ground potential(s) as the UUT
- fixed interface connector to base of bed of nails, but more expandable as more connectors can be added for more generic test jigs beyond the standard 2
- fixed number of inputs/ outputs per generic test board – scalability and configuration achieved by adding more boards if required beyond the standard 2

- All connectors to the bed of nails accessible from one side to facilitate easier swapping of beds of nails.
- Physical interlock to stop the mechanical test jig cover from being opened while a test is in progress, and also to help ensuring that the correct label is placed on the correct device when testing in parallel with a single printer (scan to ensure the device and label serial numbers are the same)
- Single processor on the generic test board and measurements done using the processor's on board peripherals
- LDR light measurement to measure led light.
- impedance test functionality.
- Connectors for external power supplies.

All Schematics and gerbers for the boards in Iteration 5 are available in Appendix F.1.

3.6.2 Detailed Design Description

The designs for the generic test jig board are described here as Iteration 5 represents the Final version of the board, and as such the most complete (previous iterations may have had some parts of the circuits only partially implemented, or implemented only to enable the debugging and testing process for the board). Below are the descriptions of the individual parts of the Generic Test Jig Board:

3.6.2.1 Power Supply

The generic test board can either be powered via an on board power supply with a 220V mains input, or using an external 12V/5V power supply unit. The Board is laid out in such a way, that if either one of these options is chosen, the other one physically cannot be implemented. For example, the tracks to the 0 Ω resistor R44 which allows the switch mode power supply to run off of the external 5V power supply, is placed in the same physical location as the bridge rectifier D6 that would normally supply the input to the switch mode rectifier. The same applies with resistors R45, R46 and connector P2 which allow the 12V and 5V external supplies to be brought onto the board are placed below the transformer T1 allowing only one option to be populated.

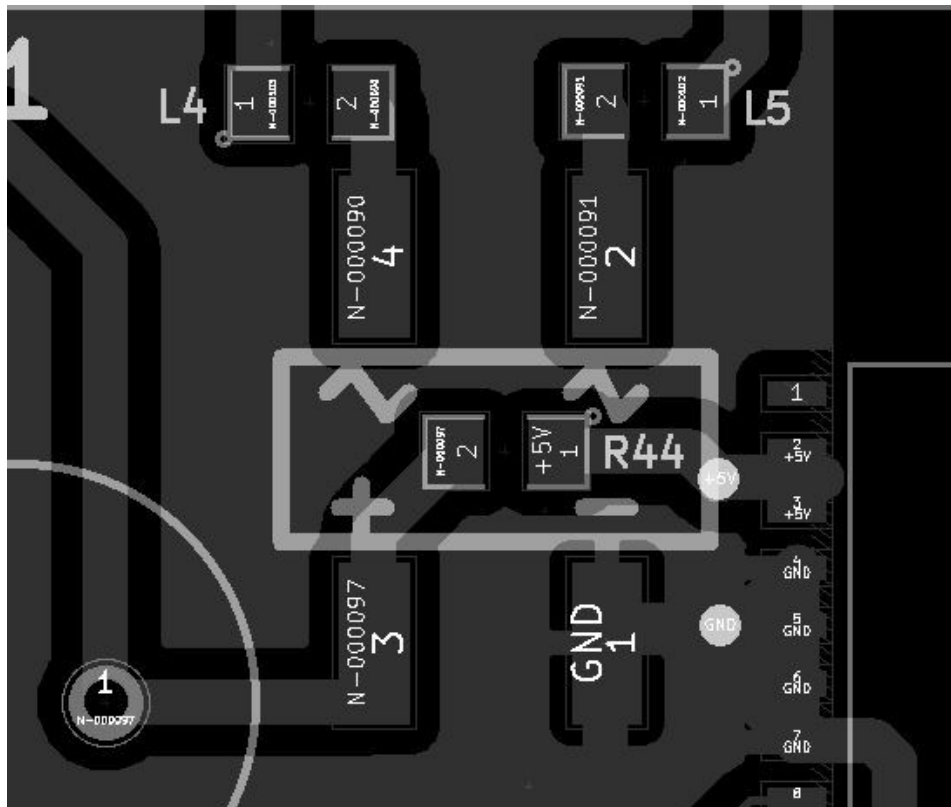


Illustration 35: Mutually Exclusive Placement of R44 and D6

The following table shows which components should be fitted depending on which power supply option is required.

Power Supply	Fit	Do Not Fit
On Board	T1 ,P1,L4,L5,D6 U4,U7,C20,C22,C24	P2 ,R44,R45,R46
External	P2 ,R44,R45,R46	T1 ,P1,L4,L5,D6 U4,U7,C20,C22,C24

Table 9: Components To Place For External vs On Board Power Supply

There are 3 voltages used in the generic test jig system:

- 3.3V used to power the controller and any logic on the test jig stack that is required. 3.3V can also be used to supply power to the UUT, provided it does not overload the power supply. The 3.3V supply is either fed from an external 5V supply, or from the on board 12V transformer.
- 5V used to power the ICoupler isolator as well as any logic specifically requiring 5V. If the on board supply is used, this should not be used to power the UUT. The 5V supply is either supplied externally, or fed from the 12V supply.

- 12V used to switch relays or to power the UUT. The 12V supply is either supplied externally or supplied from the on board transformer.

Transformer

The Transformer on the board is a Jessiva A103 230V transformer with a 12Vac output and a 5VA rating. The current that the transformer can supply is therefore:

$$\begin{aligned} I_{max} &= \frac{5\text{VA}}{12\text{V}} \\ &= 416\text{mA} \\ &\approx 410\text{mA} \end{aligned}$$

and the transformer internally fuses after this value (500mA fuse point). The output current is shared by the three power supplies. The output voltage is rectified by the DF1510S bridge rectifier (alternative bridge rectifier B380S can also be used) and the the full wave rectified voltage is smoothed by two 470uF electrolytic reservoir capacitors C20 and C21. The rectified output voltage will peak at:

$$\begin{aligned} V_{dc} &= V_{ac} \times 1.414 \\ &= 12 \times 1.414 \\ &= 16.97\text{V} \\ &\approx 17\text{V} \end{aligned}$$

+3.3V Supply

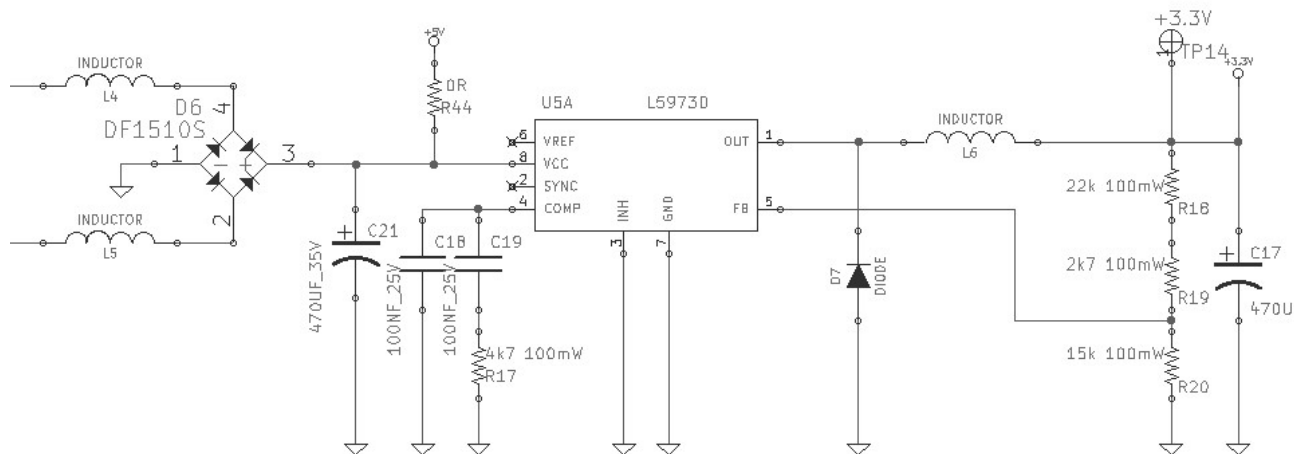


Illustration 36: 3.3V Switch Mode Power Supply

The 3.3V supply is built around a L5973D step down switching regulator. The input voltage range for the chip is from 4V to 36V [28, table 2 page 4]. If the external supply option is used, then R44 is populated and 5V is supplied to the input VCC pin of the regulator. If the on board power option is used, then the voltage is supplied from the on board 12V transformer at $\approx 17\text{Vdc}$, which is well within the 36V limit. The output of the switch mode power supply is calculated from the feedback resistors R18,R19 and R20. The expected output is:

$$\begin{aligned} V_{out} &= V_{ref} \left(1 + \frac{(R18 + R19)}{R20} \right) \\ &= 1.235 \left(1 + \frac{(22\text{k} + 2\text{k}7)}{15\text{k}} \right) \\ &= 3.27\text{V} \end{aligned}$$

Where:

V_{out} = the output voltage

V_{ref} = 1.235V reference voltage

Which with values of R18 = 22k, R19 = 2k7 and R20 = 15k gives an output voltage of 3.27V. The 470uF electrolytic capacitor on the output acts as a reservoir.

The inductors L4 and L5 in the circuit to the D6 bridge rectifier are 0805 ferrite filter inductors to stop noise from the switch mode power supply leaking back onto the mains input due to the switching frequency of 250kHz. The Flyback coil L6 was chosen as a shielded 33uH coil as this value was already a stocked value in the Util Labs store.

The only components on the generic test board that draw current continuously from the

3.3V rail are the STM32F103 microprocessor and the AT45DB32 flash chip, which draw in total < 50mA.

+12V and +5V Supplies

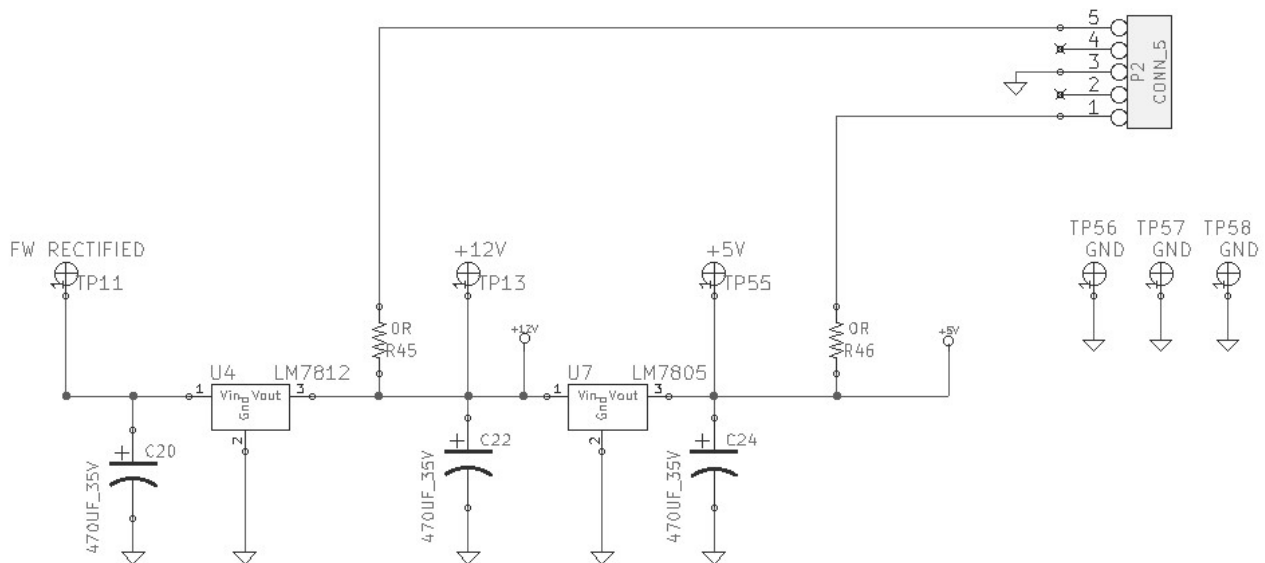


Illustration 37: The 12V And 5V On Board Power Supplies

The on board 12V supply is fed by the on board 12V transformer that supplies it with a rectified voltage of $\approx 17V$, and the 5V regulator is fed from the 12V supply. The regulators are standard LM7812 and LM7805 linear regulators in TO-220 packages. The 5V supply is supplying the ADM3251E ICoupler which draws 110mA nominally from the power supply when not connected to anything, and up to 145mA when connected to another RS232 device (dependent on the load resistance that chip places on the transmitter of the ICoupler) [24, page 3 table1]. Since the 5V chip is driven from the 12V rail, the drop across it will be 7V. For the TO-220 package, θ_{JA} (Junction to Ambient thermal resistance) is $54^{\circ}C/W$ [29, page 6 note 2]. Assuming an ambient temperature inside of the test jig of $35^{\circ}C$, then to keep the regulator junction below the maximum $150^{\circ}C$ (de-rated to $120^{\circ}C$), the maximum power that can be dissipated for both the 12V and 5V supplies is:

$$\begin{aligned}
 P_{max} &= \frac{(T_{max} - T_a)}{\theta_{JA}} \\
 &= \frac{(120 - 35)}{54} \\
 &= 1.57W \\
 &\approx 1.5W
 \end{aligned}$$

Where :

T_{max} = The maximum case temperature in °C

T_a = The ambient temperature in °C

θ_{JA} = The thermal resistance of the component junction to ambient

Since the base current (for the ICoupler) drawn from the 5V regulator can be assumed at 145mA, this current will contribute:

$$\begin{aligned}
 P_{5V} &= V_{drop5V} \times I \\
 &= 7V \times 145mA \\
 &= 1.015W
 \end{aligned}$$

Where :

P_{5V} = The dissipation accross the 5V regulator

V_{drop5V} = The voltage drop accross the regulator

I = The current drawn from the supply for the ICoupler

and

$$\begin{aligned}
 P_{12V} &= V_{drop12V} \times I \\
 &= 5V \times 145mA \\
 &= 0.725W
 \end{aligned}$$

Where :

P_{12V} = The dissipation accross the 12V regulator

$V_{drop12V}$ = The voltage drop accross the 12V regulator

I = The current drawn from the supply for the ICoupler

which means that only 0.5W can be drawn from the 5V supply and only 0.775W from the 12V supply by the test jig stack. If any more power is required than can be supplied by the on board power supplies, an external 5V/12V supply can be used as described above.

3.6.2.2 ICoupler

The ADM3252E ICoupler is the basic isolating component used throughout this project. It is a UART to RS232 converter chip that allows 2.5kV isolation between UART and RS232 sides of the chip. The main advantage of using the Coupler rather than an opto coupler arrangement is the fact that the ICoupler generates its own power supply for the isolated section by using on chip CMOS transformers

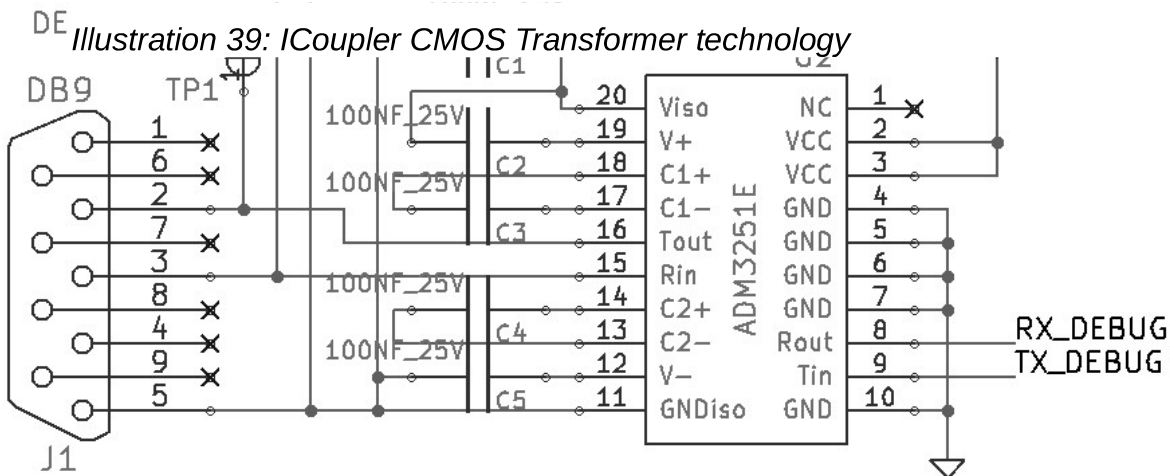
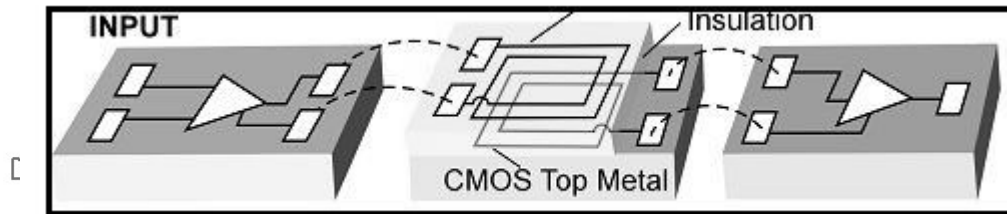


Illustration 38: ICoupler Schematic
image retrieved from [15]

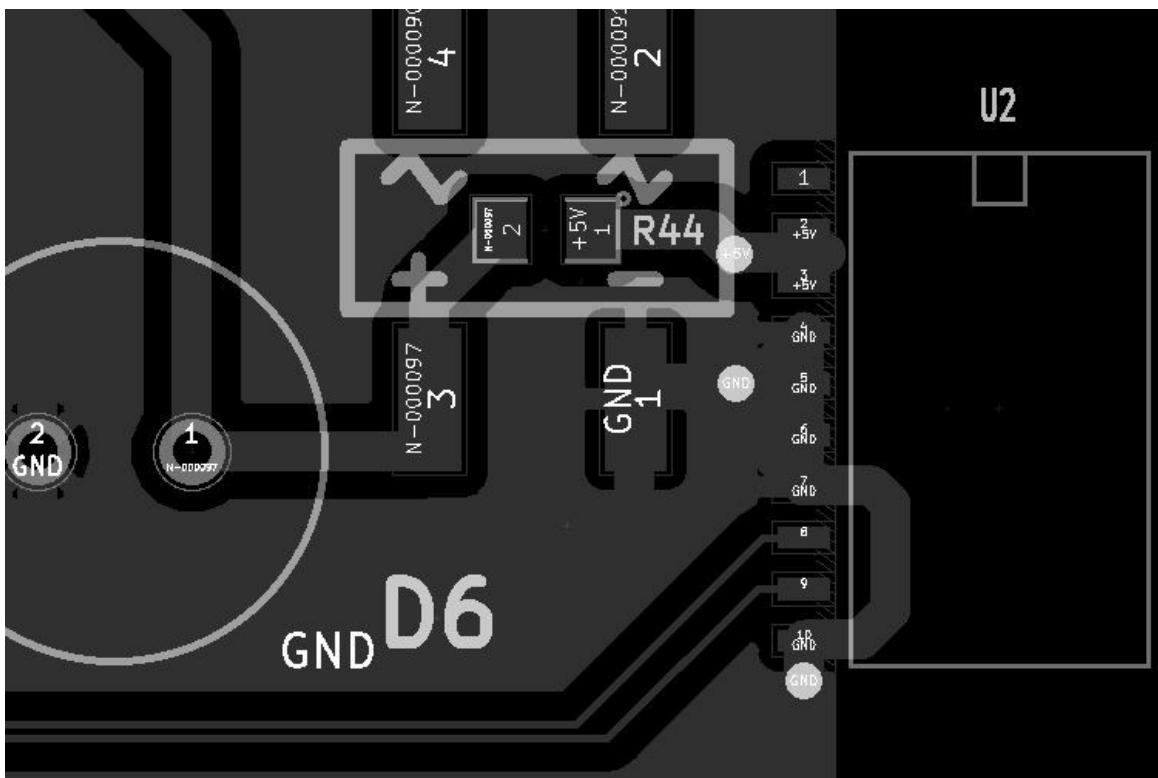


Illustration 40: Vias Added And Copper Over Pads To Increase Heat Sinc

The data is transferred from the input side of the chip to the output side by generating 1ns pulses on state changes in the input. The power is also transferred from the one side to the other by means of a 300Mhz internal oscillator that feeds a CMOS transformer and is rectified and regulated on the Isolated side of the chip^[16]. It is due to this high frequency oscillator that the chip has a fairly high current consumption and heat dissipation. To do away with excess heat, the chip has a large amount of heat sink in the form of copper around it on both the top and bottom copper layers of the PCB. Other than for the addition of one 100nF capacitor, the ADM3251E chip schematic is similar to standard RS232 converters like the MAX232. The UART RX and TX signals are fed into the chip on pin 8 and 9, and the isolated RS232 signals are available at pin 15 and 16. C1 is a smoothing capacitor for the isolated voltage that is generated by the ICoupler, and capacitors C2-C5 form part of the charge pump of the chip to create the RS232 voltages.

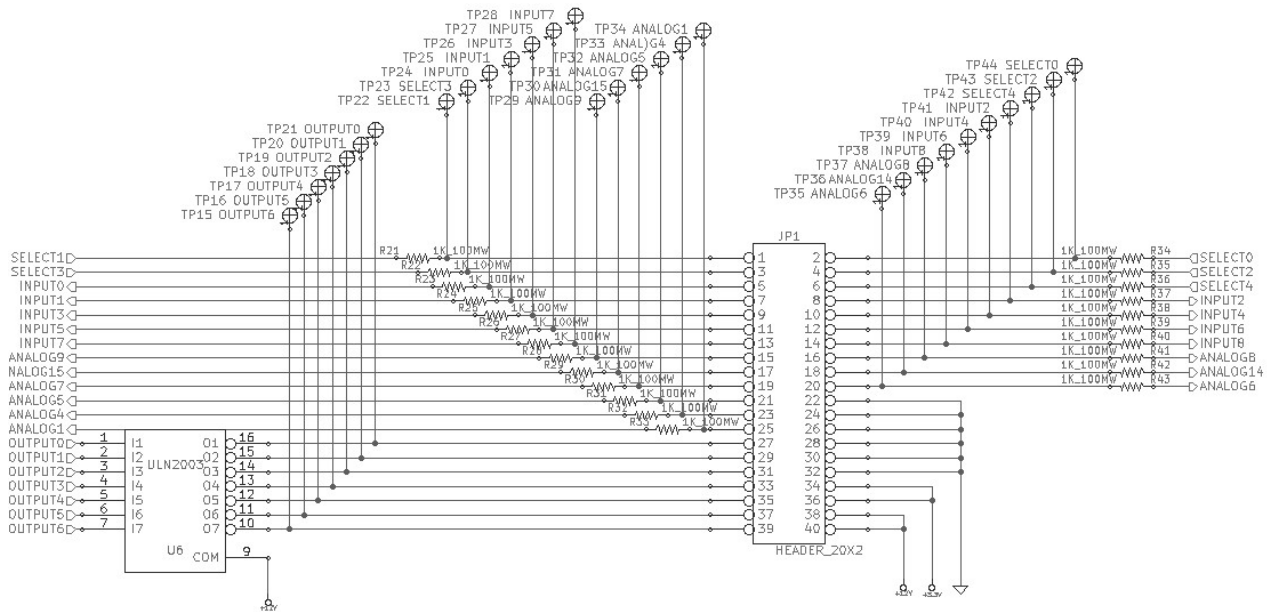
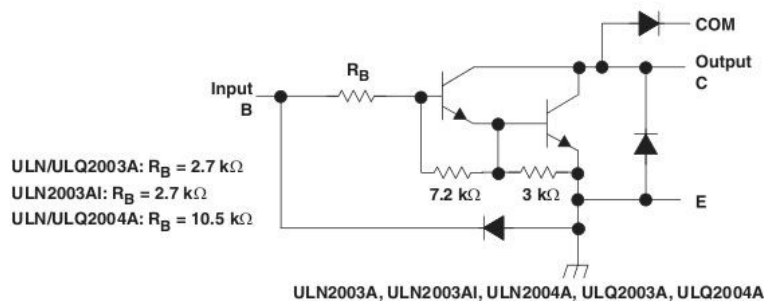


Illustration 41: The 40 pin Interface Connector And Output Driver

The outputs on the generic e jig boards are all driven through a ULN2003A Darlington driver chip. The Darlington driver already has a bias resistor to the input of the transistor pair for each channel, and a diode to the common rail (12V) allowing it to drive inductive loads up to 500mA [30, page 2 table MAXIMUM RATINGS].



All resistor values shown are nominal.

The collector-emitter diode is a parasitic structure and should not be used to conduct current. If the collector(s) go below ground an external Schottky diode should be added to clamp negative undershoots.

Illustration 42: Schematic For One Channel Of The ULN2003A driver

image retrieved from [30].

The Schematic above also describes the final 40 pin ribbon interface connector pin-outs.

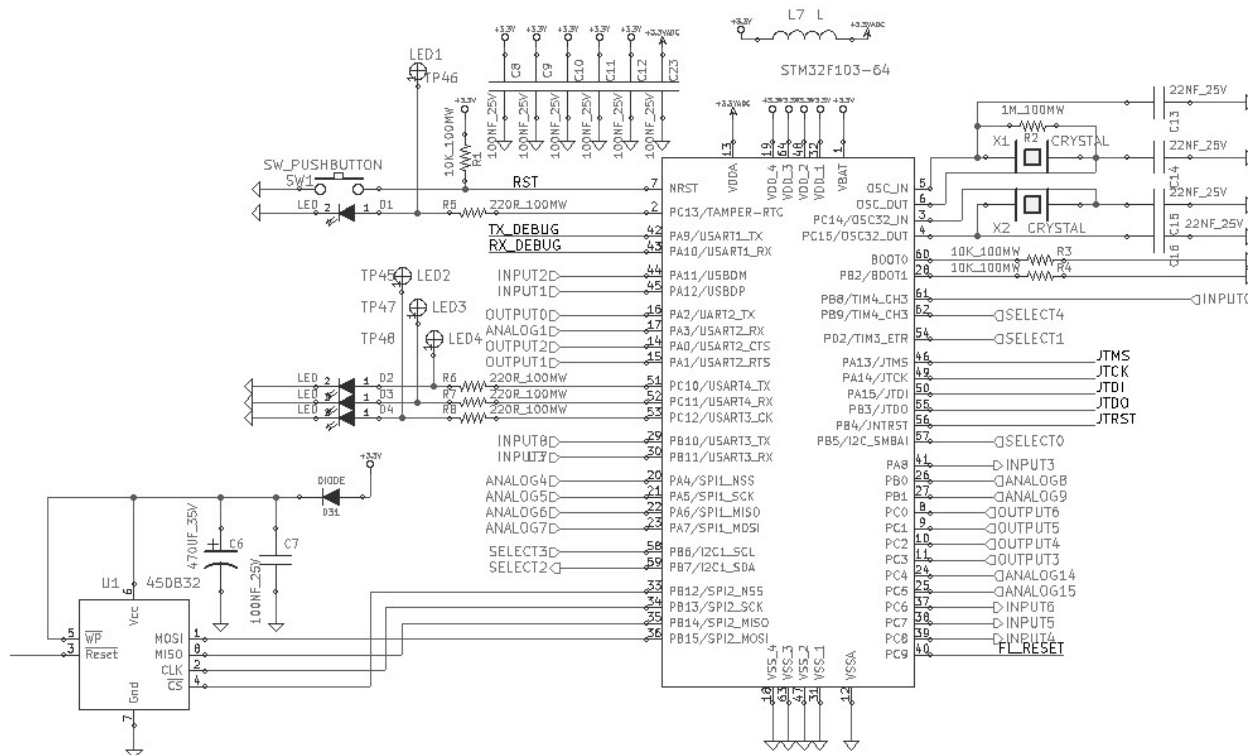


Illustration 43: STM32F103 Microprocessor Generic Test Board Controller

The main controller on the generic test jig board is an STM32F103 microprocessor. All of the measurements on the UUT are eventually done with this microprocessor's internal peripherals (through appropriate signal conditioning on the test jig stack). The AT45DB32 flash memory is a requirement of the EOS operating system. All measurements done by the controller are done on the prompting of the test jig PC through a message, and the results of the measurements are sent back to the test jig PC in a message when the results become available. The main actions performed by the controller are as follows:

Action	Request Message	Response Message
Measure voltage points	0x730	0x731
Measure logic level	0x732	0x733
Start a pulse test	0x734	N/a
Stop a pulse test	0x735	N/a
Count a number of pulses*	0x736	0x737
Measure pulse frequency*	0x736	0x737
Switch an output	0x011	N/a
*Before pulses can be measured, a pulse test must be started		

Table 10: Actions Performed By The Generic Test Jig Controller

The firmware that controls this process is described in section 3.6.3.

The controller is programmed by a standard JTAG interface.

Analog measurements are done using the on board ADC peripheral, and as such the conversion is possible across the full range from 0V to the Vdda voltage [19, page 103 table 59], tied on the board to the 3.3V rail through the LC filter formed by the ferrite bead L7 and C23. The ADC is used at the slowest possible speed (239.5 cycles per conversion) to make sure that high divider resistors in the signal path have minimal impact on the measurement. The measured value is calculated as follows:

$$V_{uut} = \frac{V_{measure} \times (R1 + R2 || R_{adc})}{R2 || R_{adc}}$$

Where:

V_{uut} = The voltage to be measured

$V_{measure}$ = The voltage measured at the ADC

$R1$ = The top resistor in the divider

$R2$ = The voltage to be measured

R_{adc} = The input impedance of the ADC voltage

The input impedance of the ADC input pins is the ADC channel resistance (R_{ain}) in series with the sampling switch resistance (R_{adc}), in series with the 1k protection resistors (R28-R33 and R41-R43). R_{ain} is given as 50k Ω and R_{adc} as 1k Ω in the datasheet

[19, page 103 table 59], which gives an input impedance of 52k Ω .

3.6.2.5 Light Sensor

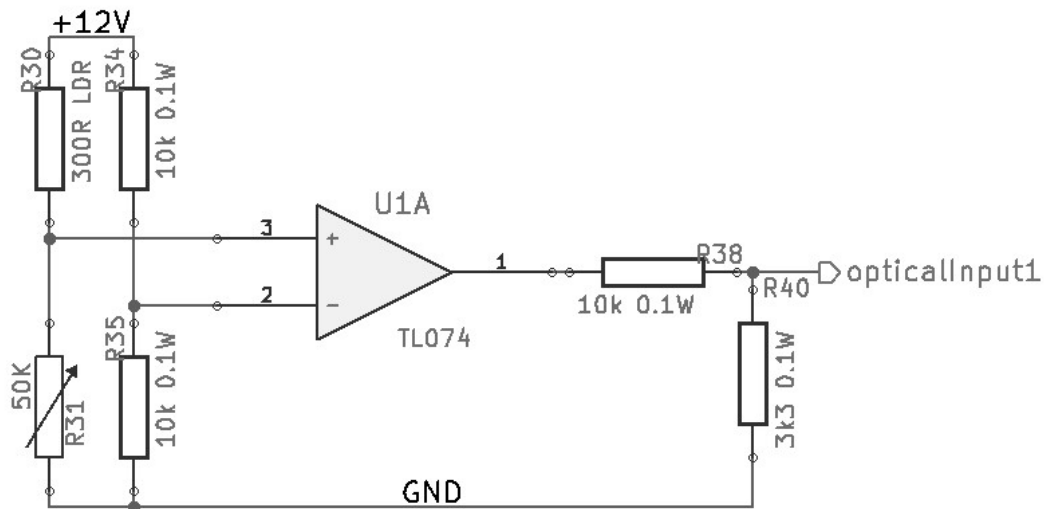


Illustration 44: Light sensor

The light sensor is placed on the stopper board is built around a Jameco 9001 light

Dependant resistor. The LDR has a resistance of 300k Ω in no-light conditions, 20k Ω at 1lux light level, and 50k Ω at 10 lux light level [27, page 2 graph k Ω vs lux]. This LDR is placed in series with a 50k Ω multi turn potentiometer which is then calibrated to the maximum light level that the test jig will see with the UUT placed in the test jig, and the light source on the UUT off. This is then used as the positive input to a TL074 op-amp that is used as a comparator_[32]. The negative input is then connected to a resistor divider created by two 10k Ω resistors, which biases the negative input at 6V (half way on the 12V rail). When there is no light source present, the potentiometer will have a lower resistance than the LDR. When the light source goes on, the resistance of the LDR drops (since the LDR is placed right under the light source of the UUT, the 1 lux level will be reached for devices with minimal lumen ratings), and the voltage on the positive input of the comparator will rise above the 6V bias level and change the state of the output. A resistor divider network is added on the output signal to drop the 12V output signal down to an appropriate level for the 3.3V inputs on the generic test jig.

Take note that the LDR has a 25ms decay time and 60ms rise time which means that pulsed signals above 10Hz will not necessarily be measured correctly.

3.6.2.6 Impedance Measurement And Power Test

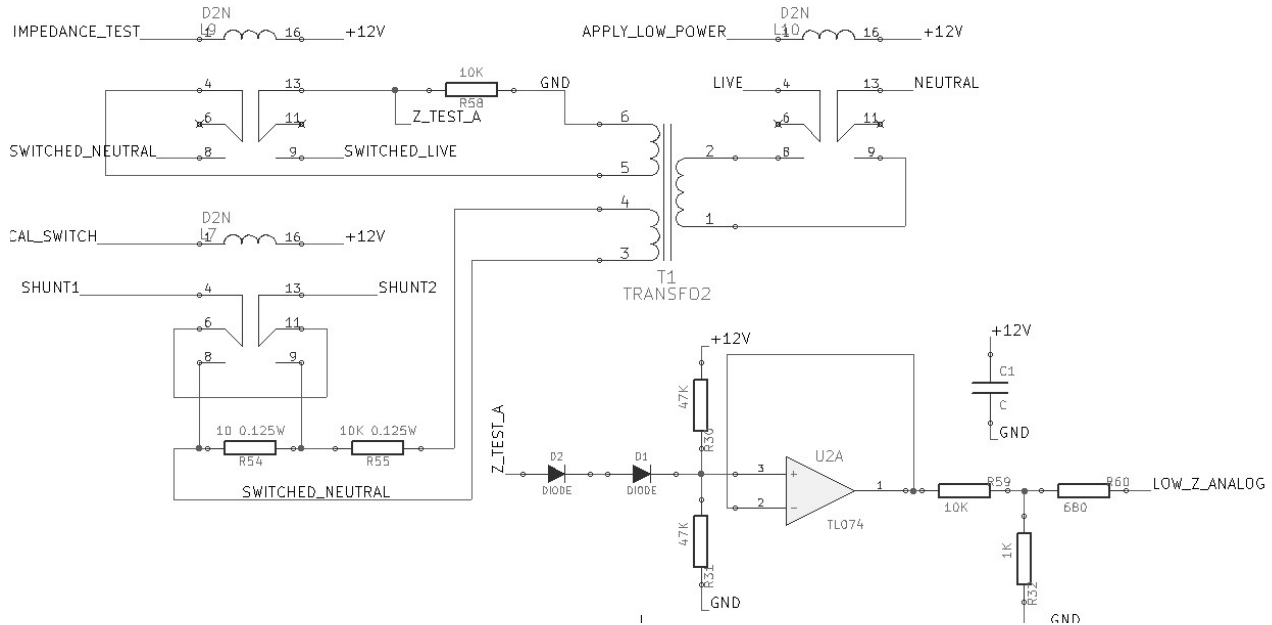


Illustration 45: Impedance Measurement Circuit

Impedance Measurement

The impedance measurement hardware is placed on the Base board of the test jig stack. The impedance measurement test is run before any of the other tests on the UUT to check if there is either a short circuit or an open circuit across the voltage input pins of the UUT. In the case of the MCU guinea pig board, the power input to the board is 220V mains. The circuit is placed into impedance measurement mode by actuating the LOW_POWER and IMPEDANCE test relays (L10 and L9 respectively) which powers the low power transformer T1 and supplies 12Vac to the LIVE and NEUTRAL input pins to the UUT. The 12Vac is in series with a 10k Ω resistor R58. A resistor divider is now formed between the impedance of the UUT and the 10k Ω resistor. The output of this resistor divider is now rectified through diodes D1 and D2 which both rectify the signal and drop it by 1.4V to ensure the TL074 op-amp U2 is not damaged. The op amp is normally biased at 6V through the resistor divider of R30 and R31, but the current flows through the rectifier diodes to ground through R31, this will cause the bias voltage to vary. If there is no UUT board present, or the board measures open circuit, the input voltage on U2 will remain at 6V, but if there is a short circuit, then R58 will see the full voltage of T1 and this will raise R31 closer to the 12V rail. U2 acts as a buffer, with the output going to a resistor divider formed by R59 and R32 which will drop the buffered output of the op-amp to a suitable level for input into the generic test jig. An additional 680 Ω series resistor R60 is added to offer additional protection to the input of the generic test jig.

Suitable impedance values for production boards are found experimentally by measuring the impedance of a number of known good boards and setting appropriate boundaries for the measured voltage.

Power Test

Since the MCU guinea pig board has a function to measure series current, a power test circuit was added to force an amount of current through a mock shunt R54 which will mimic the 0.001 Ω on the UUT and test that power level is measured to ensure the shunt and power measurement circuit is working correctly. A 10 Ω mock shunt is used, as it will simulate a much higher current, since the voltage drop across a 10 Ω resistor will be 10000 times higher than the equivalent current through the shunt resistor.

The same transformer is used for the low voltage source as for the impedance test, only this time the CAL_SWITCH and APPLY_LOW_POWER (L7 and L10 respectively) are

actuated. The mock shunt is placed in series with a 10kΩ resistor R55 which causes a current of:

$$\begin{aligned} I_{mock} &= \frac{17V}{10 + 10k} \\ &= 1.698mA \\ \text{and} \\ V_{mock} &= I_{mock} \times 10 \\ &= 16.98mV \end{aligned}$$

Which simulates a current of:

$$\begin{aligned} I_{sim} &= \frac{V_{mock}}{R_{shunt}} \\ &= \frac{16.98V}{0.001\Omega} \\ &= 16.98A \end{aligned}$$

This simulated current is measured by looking at the output of the measurement chip on the UUT which has a pulsed output corresponding to the measured amount of current. During the test, the measurement chips are not yet calibrated on the UUT, so there is a wide range of acceptable values for the pulse frequency, but it must send pulses to pass the test.

3.6.3 Firmware

The firmware on the generic test jig allows the actions described in section 3.6.2.4 to be performed by the hardware. To understand the firmware sections, some background on the EOS operating system is required. Information on the operating system is available in Appendix G. The following files are important to the functioning of the generic test jig in iteration 5:

- gw_FastInputs.cpp and gw_FastInputs.h
- gw_Adc.cpp and gw_Adc.h
- gw_TestJigGpio.cpp and gw_TestJigGpio.h
- drv_FastInputs.cpp and drv_FastInputs.h
- drv_Adc.cpp and drv_Adc.h
- tsk_StateTracker.cpp and tsk_StateTracker.h

The individual files are available in Appendix F.2. All of the classes are discussed in section 3.6.3.4.

3.6.3.1 Inputs and Outputs

Relevant files: tsk_StateTracker.cpp, tsk_StateTracker.h

Relevant Messages: “State Changed” (0x10), “Set State” (0x11), “State Request ” (0x732), “State Response”(0x733)

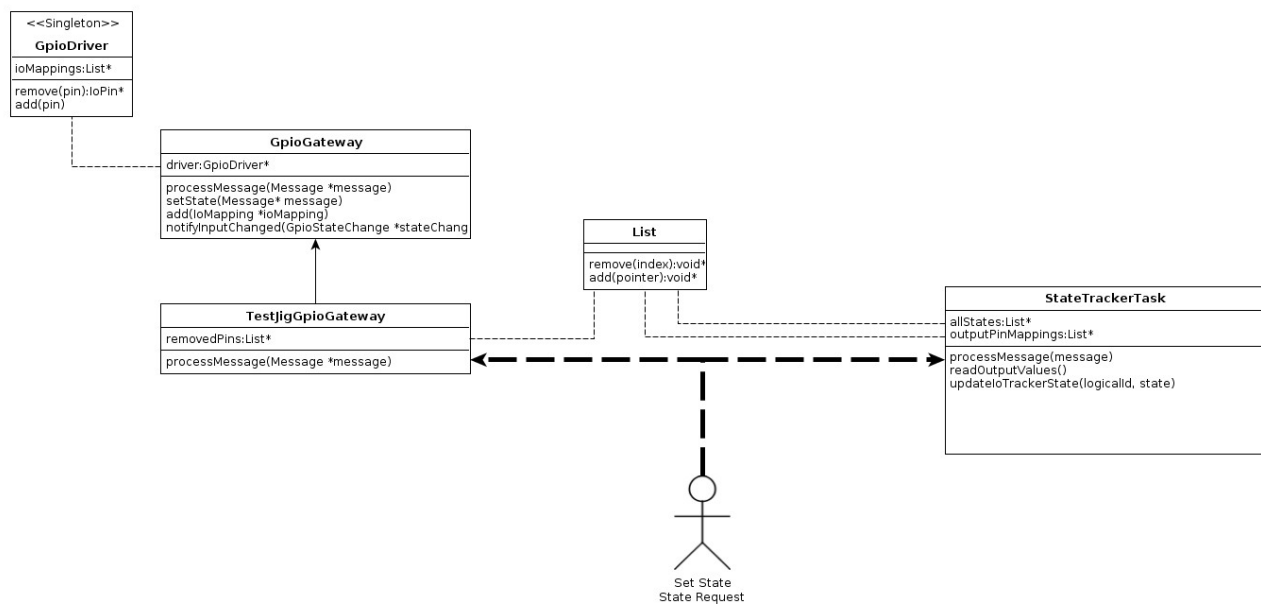


Illustration 46: Input and Output Actions (Only Relevant Methods and Members Shown)

Physical State Change

Whenever a physical pin that is tracked by the GpioDriver changes state, the system is notified of it by the TestJigGpioGateway sending out a “State Changed” (0x10) message. The problem with this, is that the test jig PC may not have been listening when that message was sent as it is not assumed to be connected to the internal message bus of the generic test jig system 100% of the time. So the StateTrackerTask will intercept this state, and track it along with all of the other inputs on the generic test jig.

“Set State” (0x11)

When a set state message is received, the TestJigGpioGateway will instruct the GpioDriver to change the state of the output to the state specified in the “Set State” message

“State Request” (0x732)

When a “state request” message is received for an input on the system, the StateTrackerTask will respond with a “State Response” (0x733) message if the LogicalId specified in the message in its list of known states, or it will ignore the message, assuming it is for another task if it is not in the list.

Using the messaging interface for the switching of IO on the generic test jig allows the test jig to comfortably slot into the existing product line of Util Labs. A lot of code re-use is possible as all system interactions in the ULM system are message based.

3.6.3.2 Fast Inputs

Relevant files: gw_FastInputs.cpp, gw_TestJigGpio.cpp, drv_FastInputs.cpp, gw_FastInputs.h, gw_TestJigGpio.h, drv_FastInputs.h

Relevant Messages: “Start Pulse Test” (0x734), “Stop Pulse Test” (0x735), “Pulse Data Request” (0x736), “Pulse Data Response” (0x737)

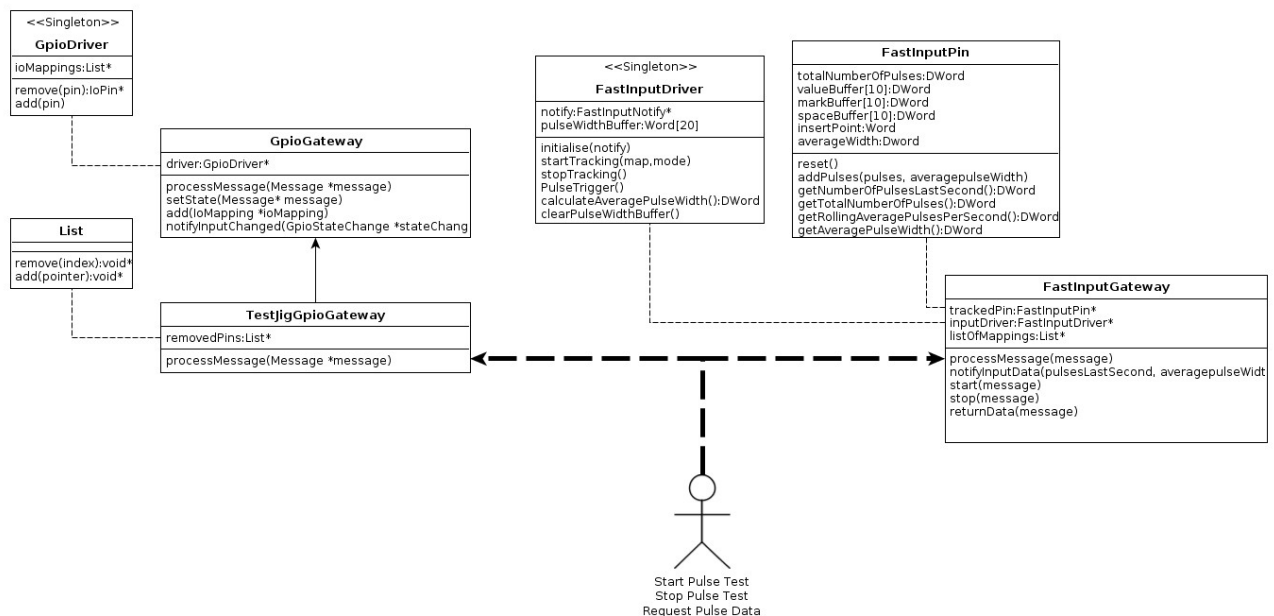


Illustration 47: Pulse Test Actions (Only relevant methods and members)

“Start Pulse Test” (0x734)

When the “Start Pulse Test” message is received by the system, the TestJigGpioGateway disables the appropriate input pin in the and GpioDriver so that it is no longer tracked and debounced. At the same time the FastInputGateway receives the same message (since they are on the same message bus) and instructs the FastInputDriver to set the pin to an

external interrupt, with the appropriate edge detect mode (rising, falling or both), and then the gateway creates a FastInputPin object that will track all of the information for that pin during the test. While the test is running and there is a FastInputPin active, the FastInputDriver will notify the FastInputGateway once a second of the number of pulses received as well as the average pulse length of those pulses. This information is passed from the gateway to the FastInputPin So it can be tracked

“Request Pulse Data” (0x736) and “Pulse Data Response” (0x737)

If there is no active FastInputPin and a “Request Pulse Data” message is received then the gateway will respond with a “Pulse Data Response” that has LOGICAL_ID_INVALID (0xffff) as its pin Id specifier. However while the pulse test is active, and there is an active FastInputPin requests will be responded to with valid pulse data.

“Stop Pulse Test” (0x735)

Once all of the required data has been collected, a “Stop Pulse Test” message is sent by the test jig PC. Once this message is received, the TestJigGpioGateway will return the pin it removed from it to the GpioDriver which can then once again resume monitoring and debouncing it. At the same time the FastInputGateway will instruct the FastInputDriver to reset the required pin back to a standard input. The FastInputPin is destroyed.

The fast input mode allows all of the inputs on the generic test jig to also act as pulse capture inputs. It also allows for a lot of different tests to be run, as FastInputPin tracks not only the average pulse frequency but also the last second's pulse frequency (allowing changes in frequency to be detected), the total number of pulses since the start of the test and the average pulse width, all of which is contained in the “Pulse Data Response” message. By decoupling the tracking of the data from the hardware pin driver, it becomes very simple to later on add more functionality to the pulse driver by simply adding another message and expanding the FastInputPin tracker.

3.6.3.3 ADC

Relevant files: gw_Adc.cpp, drv_Adc.cpp, gw_Adc.h and drv_Adc.h

Relevant Messages: “Request ADC Measurements” (0x730), “ADC Measurement Response” (0x731)

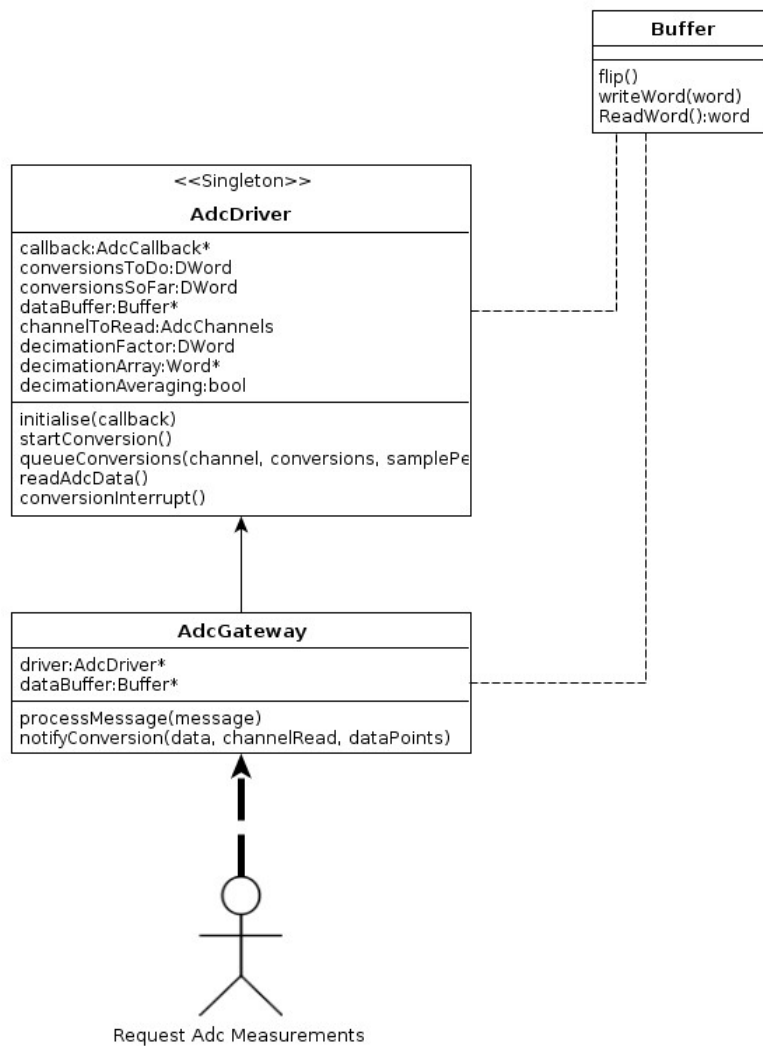


Illustration 48: ADC Test Actions (Only relevant methods and members)

“Request ADC Measurements” (0x730)

When a “Request ADC Measurements” message is received by the AdcGateway, it will instruct the singleton AdcDriver to queue the required number of ADC conversions on the requested ADC channel. The samples are done one at a time, and then once the required number of samples has been reached, the drive will notify the AdcGateway with the data. When the data is received back from the AdcDriver, the AdcGateway will pack it into a “ADC Measurements Response” (0x731) message and send it onto the message bus. To ensure that each set of ADC measurements fits into a standard message frame, no more than 100 measurements should be done at a time.

The sampling speed of the ADC is fixed at 239.5 cycles per conversion to ensure that high impedance signal conditioning circuitry will have minimal impact on the measurements

made by the ADC due to there being insufficient time to charge up the capacitor for the sample and hold circuit on the peripheral. In the “Request ADC Measurements” message the option is also given to implement decimation filtering. Decimation is a 2 step process of first applying a low pass filter to the input signal and then downsampling the data [17]. If the option is chosen to decimate, then the low pass filter needs to be applied on the test jig stack in hardware for the appropriate frequency, and the ADC driver will handle the downsampling.

3.6.3.4 Class Summary

The classes and their interactions are as follows:

- **GpioDriver**
Is the standard GPIO driver in the EOS system. It is controlled by the GpioGateway and can be instructed to switch any outputs in its list of pin mappings. It will also notify the GpioGateway when any of the pins in it's list of pin mappings changes state. The driver is internally debounced to only check the state of the pins ever 20ms. The GpioDriver is a singleton class (it can only be instantiated once)
- **List**
Is a generic list of objects and part of the EOS library. Objects added to or removed from the list are cast as void pointers to allow the list to be used on any kind of object.
- **GpioGateway**
Is the standard GPIO gateway in the EOS system. It sends out notifications when any states change on any of the pins monitored by its GpioDriver, and will instruct the driver to switch an output to a different state if it receives an appropriate message to do so.
- **TestJigGpioGateway**
A descendant of the GpioGateway used specifically on the generic test jig that allows pins to be removed from the GpioDriver when it received a “Start Pulse Test” Message, and then to return the pin to the GpioDriver when it receives a “Stop Pulse Test” message. The pins removed in this way are stored in a generic list.
- **FastInterruptDriver**
The fast interrupt driver will set up a hardware pin to fire an interrupt when it changes state when it is instructed to do so by the FastInputGateway. The pin can

be set to fire on the rising edge, falling edge or both edges. It also tracks the width of pulses that are received (the time difference between rising and falling edge) in a rolling buffer to calculate the average pulse width. The external interrupt will call a static function in the FastInterruptDriver class, and as such the driver is created as a singleton (the static method calls the object through a static pointer member). Once a second (if there is an active pin being tracked) the driver will notify the FastInputGateway of the amount of pulses recorded over the second, and the average pulse width during the second.

- **FastInterruptPin**

The fast interrupt pin object tracks all information about a pin that is currently under test as a fast input. Every time the FastInterruptDriver notifies the FastInputGateway of new pulse information (once a second), this information is passed along to the FastInterruptPin object which will add this data to any previous data. It can then be queried at any time for the pulse width since inception, average pulses per second since inception, average pulse width since inception etc.

- **FastInputGateway**

The FastInputGateway owns the FastInputDriver, and when it receives a “Start Pulse Test” message will instruct the driver to set up the required pins external interrupt with the correct mode, and then creates a FastInputPin object to track the data. When the Gateway receives a “Stop Pulse Test” it will instruct the FastInputDriver to reset the required pin back to a normal input, and will destroy the FastInterruptPin object. During the test, if a “Request Pulse Data” Message is received, the gateway will populate a “Response Pulse Data” message with all of the information tracked by the FastInputPin.

- **StateTrackerTask**

The StateTrackerTask sits on the message bus and listens for messages from the GpioGateway for when input and output pins change state. When a “State Change” message is received, it will update a list of known LogicalIds and their respective states. On receipt of a “State Request” message, the StateTrackerTask will look for the LogicalId in the request in its list of tracked values. If the LogicalId is found, then the state is sent out in a “State Response” message, however if it is not found, then nothing will be sent and the request will be ignored (assuming the message is for some other task)

- **AdcDriver**

When called to do so, the AdcDriver will queue a number of conversions on a specified ADC channel. When all of the conversions have been done, the data is packed into a Buffer and sent to the Notify object (in this case the AdcGateway). The Adcdriver also handles the downsampling for a decimation filter if it needs to be implemented. The AdcDriver is implemented as a singleton.

- **AdcGateway**

The AdcGateway will wait for a “Request ADC Measurements” message , and when it receives it will instruct the AdcDriver to queue the required number of conversions on the required ADC channel. Once the conversions have been done, the driver will notify the AdcGateway of the data, at which point the gateway will pack the data into a “ADC Measurements Response” message and send it.

3.6.4 Test Jig Creation Process

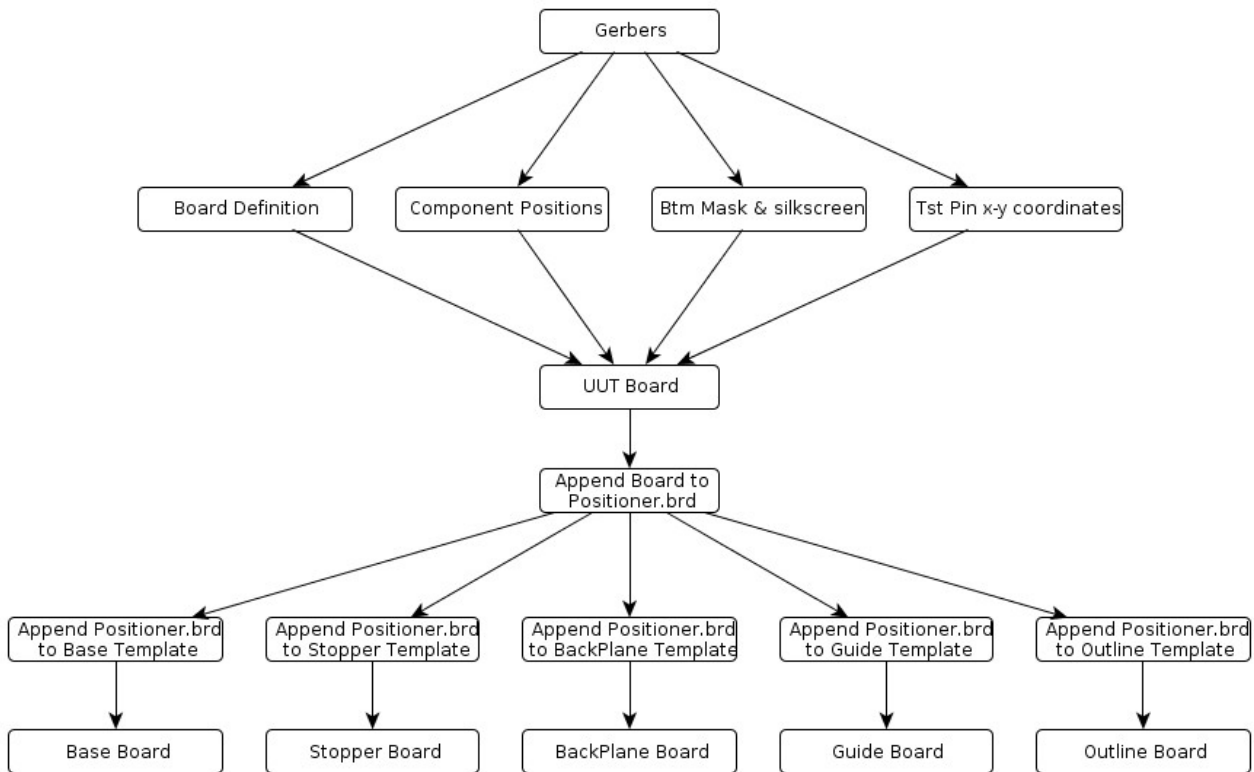


Illustration 49: Iteration 5 Test Jig Stack Creation

The test jig stack creation in iteration 5 further optimised the creation of test jig stacks by building on the work done in iteration 4 around the x-y box placement technique as well as the partially pre-routed template boards. In this iteration, instead of forcing the designer to

place the required board to test's gerbers into a .brd file, and then moving it to predetermined x-y coordinates, a “positioner” board has been created. The positioner board is not a PCB at all, but a virtual indicator of where a UUT can be placed to ensure that it does not interfere with anything. To prepare the board to be tested for positioning, the gerbers are pulled into a .brd file using gerbview:

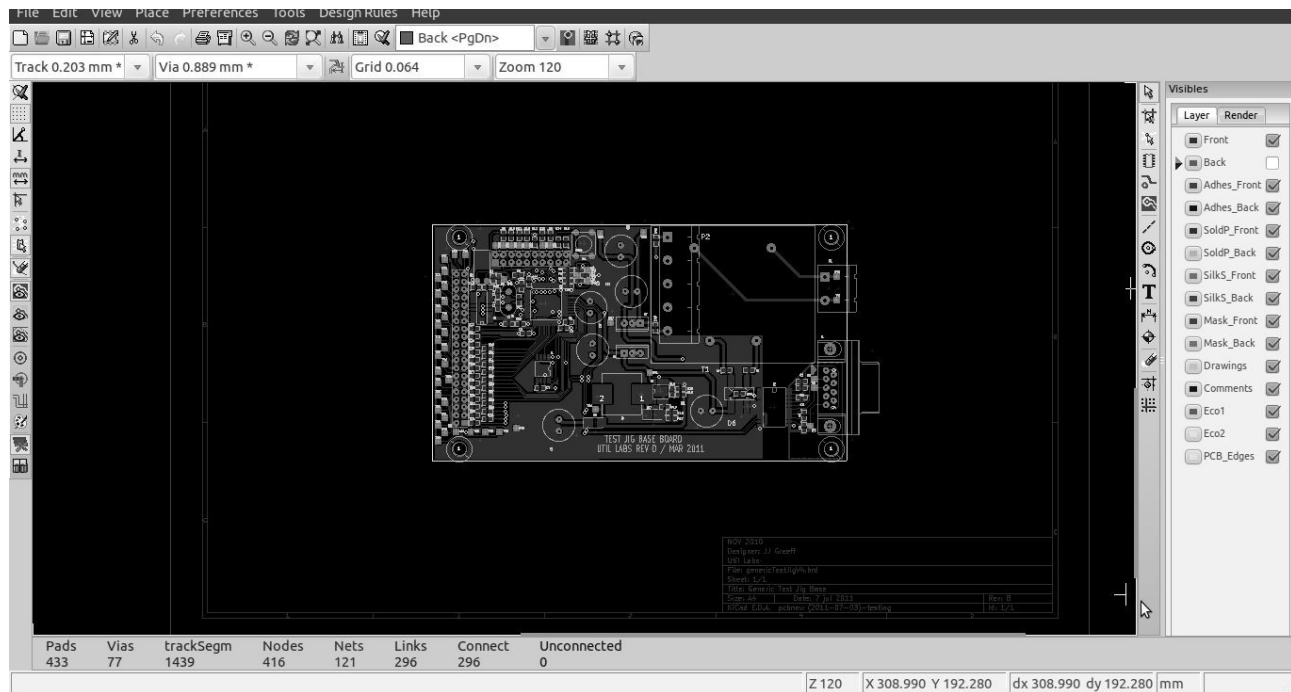
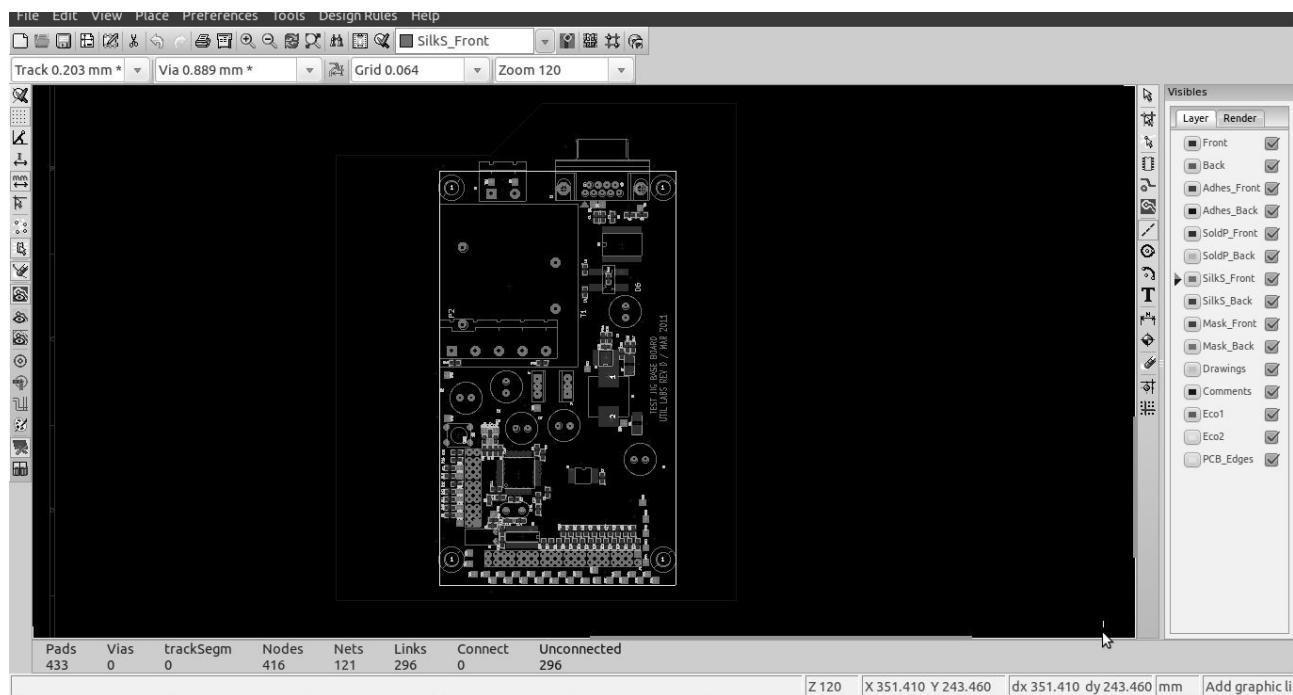
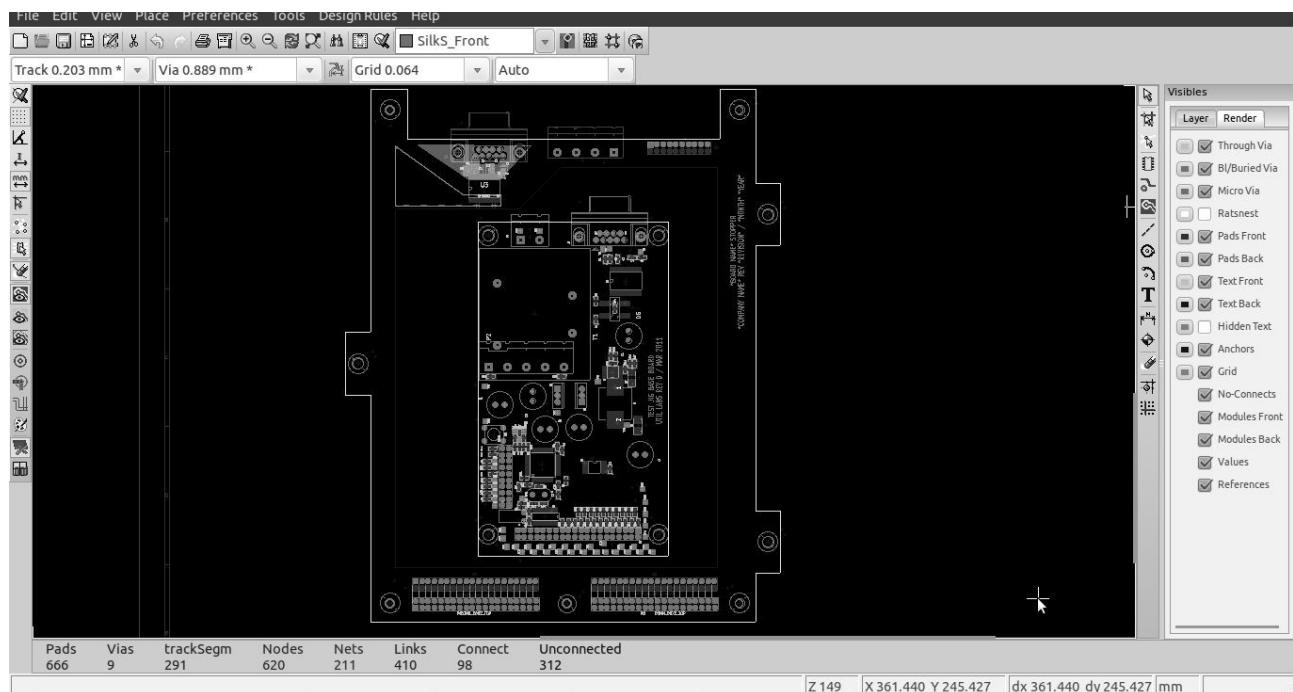


Illustration 50: Board To Test Created As .brd File

This board is now appended to the positioner board, positioner.brd, and all of the copper tracks are removed by using Edit->Global Deletions->Delete Tracks and Edit->Global Deletions->Delete Zones. The board is now positioned within the blue outline of the positioner on the board so that no part of the board protrudes, and all of the required test point objects are placed, as well as sufficient 3.15mm holes close to the test points to act as positioners for stabilising pillars between the stopper and backplane boards:



And once the board has been placed in the positioner, it can be appended to each of the 5 template boards (base.brd, stopper.brd, backplane.brd, guide.brd and outline.brd respectively) in turn:



Using this technique, all mechanical alignment issues are eliminated in the stack – provided the correct templates are used, the boards will align correctly. From there, a similar process is followed to Iteration 4 to complete the 5 boards in the stack.

- **Base Board**

Using the Base-append.brd board created by appending the positioned board, the base board is created by removing everything but the board outline (which is converted to the top silkscreen layer) and the test point positions, so that the schematic can be created (using the base template schematic) that just routes the correct lines from the generic test jig boards to the stopper board where the test points are housed through some intermediate signal conditioning placed on the base board. The pin assignments of P1 and P2 are the same as iteration 3 and 4 and can be found in table6 in section 3.4.2. Once the schematic is created, it can be pulled into the base-append.brd board and laid out taking into account the positions of the test pins so that no tall components are placed below them, as they can interfere with the assembly of the test jig stack. Once the board has been laid out, these test pin positions can be removed. The Base-append.brd board designators need to be updated to show that the board is now a base board for a specific UUT board.

- **Stopper Board**

The stopper board is created using the Stopper-append.brd created above. The outline of the UUT board is created as a silkscreen and all of the other components are removed (assuming there are no components mounted on the bottom of the test board that are higher than 14mm, as these will require cut outs on the stopper board). The schematic can be created by taking the stopper schematic template and adding the test point schematic objects to it. Any additional signal conditioning not done on the base board (like for example placing transducer hardware to read light as opposed to electrical signals) is now added to the schematic. To communicate with the UUT via a debug port, an ICoupler is added to ensure that the electrical isolation is maintained between the UUT and test PC. The ICoupler is pre placed on the Stopper.brd template with an appropriate amount of heat syncing and 4 line stubs that are used to connect +5V,gnd,RX and TX to the ICoupler. To power the chip either a pre-laid schematic of a switch mode power supply can be used, that must then be laid out around the test pins, or +5V can be brought up from the base board. If the power supply is brought up from the base board, then the U1 circuit can be removed from the schematic as in Iteration 4. The stopper board also contains the presence switch which is just a micro switch that needs to press against the UUT when the test lever is actuated on the test jig. A net just

needs to be assigned to the PRESENSE line on the schematic template.

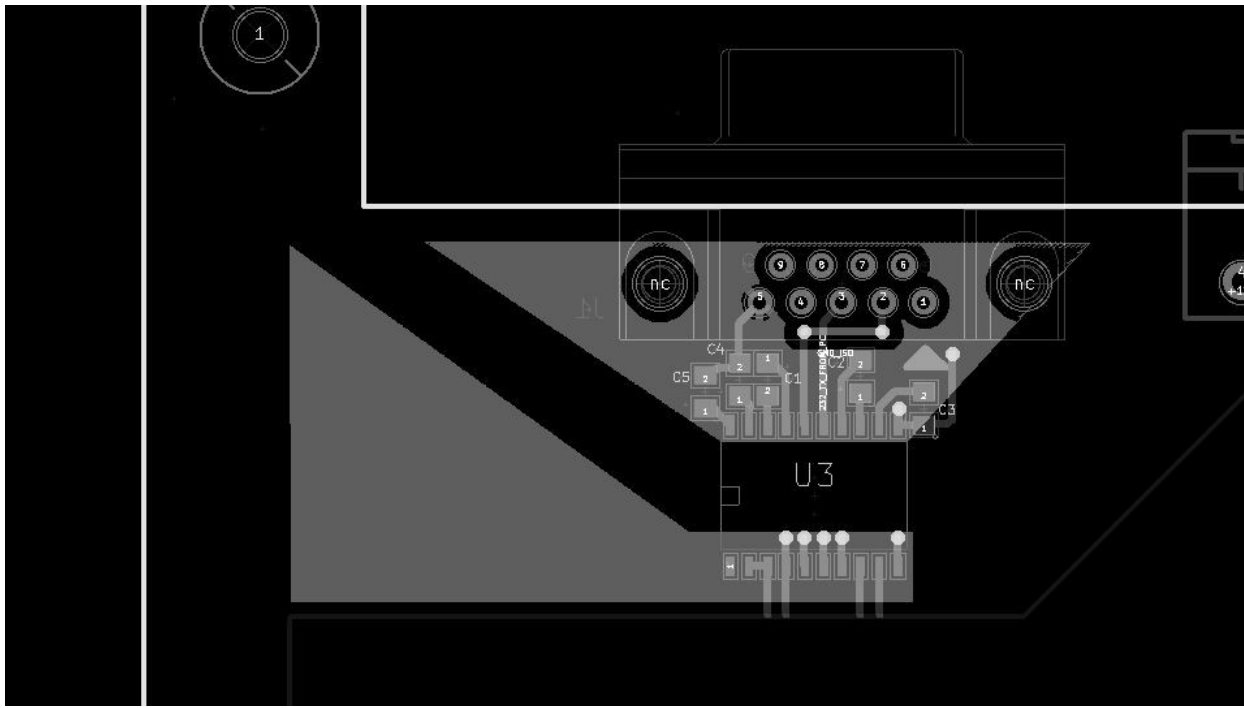


Illustration 51: Close Up Of Heat Sync And Stubs On ICoupler On Stopper Template

Once the schematic is created, the netlist can be pulled into the board and laid out, ensuring not to place any of the signal conditioning components too close to the stabilising screws for the support pillars between the stopper and backplane boards, and placing the presence switch micro switch somewhere that does not interfere with any other components when it is actuated. The Stopper-append.brd board designators need to be updated to show that the board is now a stopper board for a specific UUT board.

- **Backplane Board**

The backplane board is created from the backplane-append.brd created above. The backplane board serves the same purpose as the 2nd stopper board used to perform in iteration 4 in that it offers stability to the assembly, and creates (together with the stopper board and support pillars) a rigid plate for the test pins to press against during the test. To create the backplane board, the imported outline for the board is moved to the silkscreen layer, and all components but the test pin holes and the stabilising holes are removed from the board. There is no schematic for the backplane board and therefore no tracks, and no components are fitted onto it. When manufacturing the backplane board, only the top silkscreen layer need be sent along with the board outline layer so that the manufacturer knows not to add

any copper. The backplane-append.brd designators need to be updated to show this is now a backplane board for a specific board.

- **Guide Board**

The guide board is created using the guide-append.brd created above. The outline of the board to test is created as a silkscreen, and then the bottom silkscreen of the board is used to create cut outs for any components mounted on the bottom of the board, and the test points and through hole lead positions of other components on the top layer of the UUT are used to create holes for those leads to go through. All of the components on the board are then removed. There is no schematic for the guide board and therefore no tracks, and no components are fitted onto it. When manufacturing the Guide board, only the top silkscreen layer need be sent along with the board outline layer so that the manufacturer knows not to add any copper. The guide-append.brd designators need to be updated to show this is now a Guide board for a specific board.

- **Outline Board**

The outline board is created from the outline-append.brd board created above. The only step required for making an outline board is to draw in the Board definition layer an outline around the board to test at a distance of 0.5mm away from it and then removing all of the components on the board. The board designators need to be updated to show this is a outline board for a specific UUT board. The outline board has no schematic or copper layers.

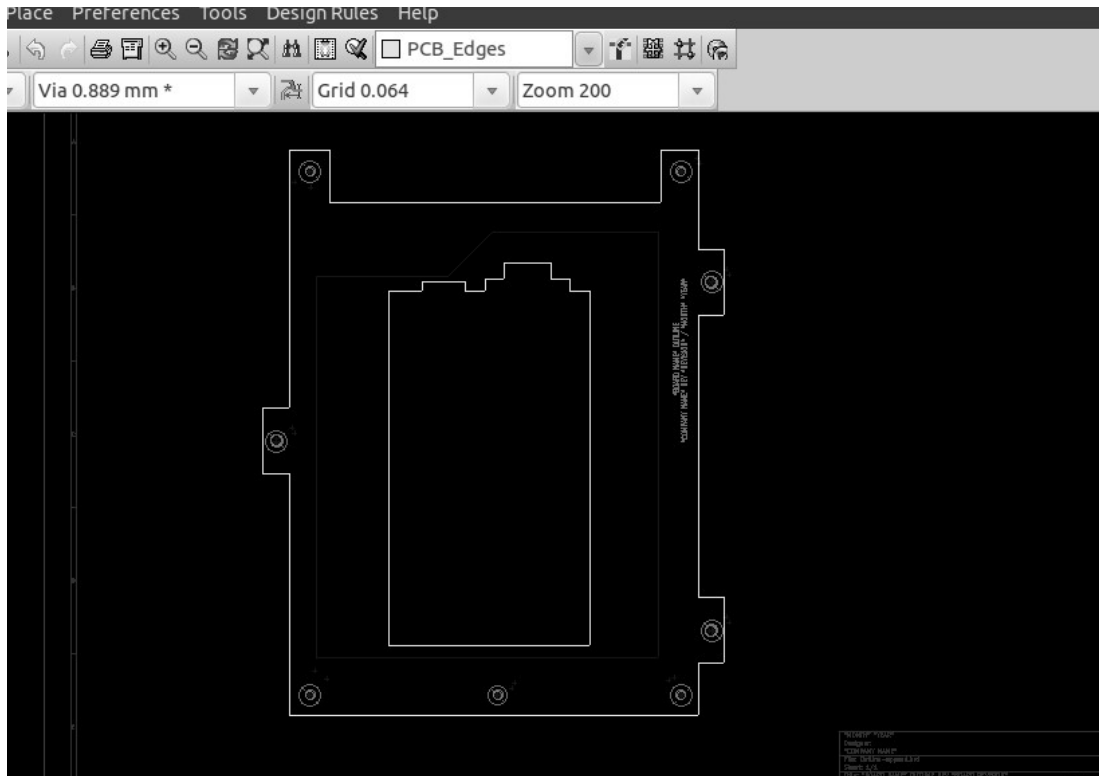


Illustration 52: Outline Board Created Using Positioner

3.7 THE JIG MECHANICS

There were two versions of the test jig mechanical base, an initial version created around the 3rd and 4th iterations of the jig, and then a 2nd mechanical version around with the 5th iteration of the generic test jig system was built. The reason for the change was that the first version was designed in house with a number of mechanical components that would have needed to be custom made, and in the end a previously designed jig that had a similar lifting plate arrangement was found that would work out significantly cheaper than the in house design. The hardware housing of the jig allows the test jig stack to be moved up and down using an actuating arm on the outside of the box, and it is this action that will cause the test pins to either make or break contact with the UUT depending on the position of the lever.

The lifting plate houses all of the boards of the generic test jig system with the exception of the outline and guide boards (they are mounted on the top plate of the jig so the operator can position the UUT), so when the arm is actuated the whole assembly moves up and down so as to minimise strain on any sensitive components on the stack. All of the wiring for the test jig stack can be done from one side of the plate, which is held in place with 8x4mm screws. When swapping between different test jig types, the technician need only drop the lifting plate by removing the 8 screws, and then swapping the test jig stack, outline and guide boards. As templates are used, the wiring for all of the test jig stacks is identical.

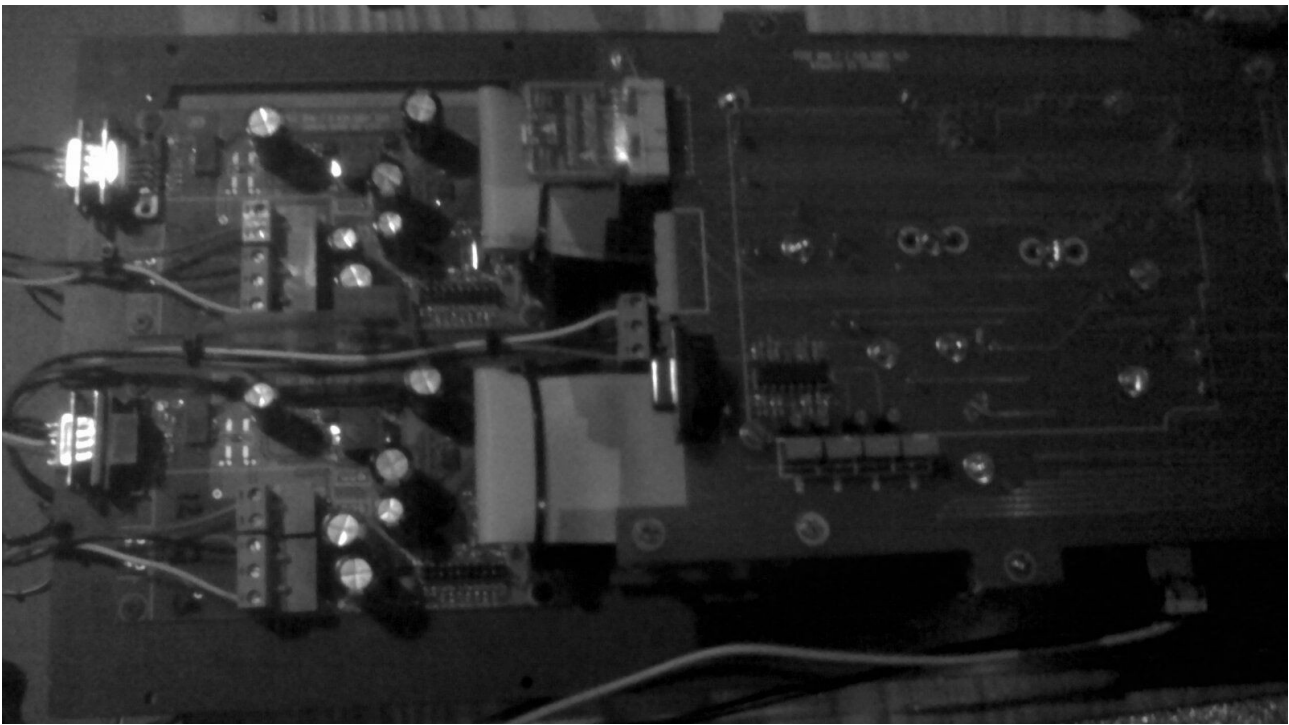


Illustration 53: Generic Test Boards and Test Jig Stack Bolted To Lifting Plate

The mechanical jig also houses some components to assist the generic test jig system in the test process:

- 3 LEDs are added to the front panel of the mechanical jig that are always wired to the stopper board which indicate RUNNING, PASS and FAIL states (orange, green and red respectively)
- 2 power supplies are mounted in the base of the mechanical jig which can supply power to the generic test jig boards.
- The mechanical counter indicating the lifetime of the mechanical jig and the mechanical interlock stops the operator opening the lid of the jig while the test is running, both of which are wired to the base board.
- The lid of the mechanical jig has a number of spring loaded pillars that place downwards force on the UUT when the test pins are pressed against the board to keep it in place.

3.8 TEST PC JAVA SOFTWARE ARCHITECTURE

The application that runs on the test jig PC houses the Java framework as well any implementations that have been done for boards that have generic test jigs created for them. To add a new jig implementation to the framework, the architecture of the framework must first be understood.

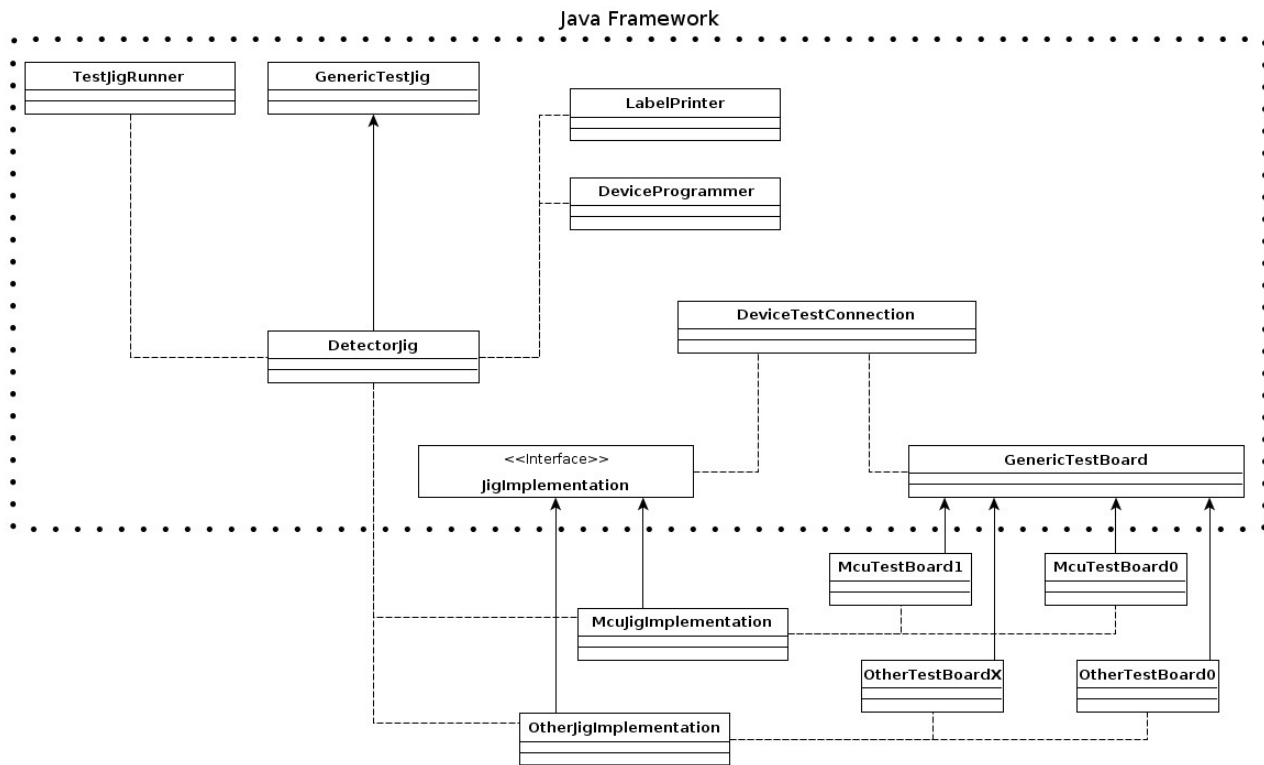


Illustration 54: The Java Architecture Overview

The main classes that need to be taken into account when writing a new test jig implementation are:

- **JigImplementation interface**

This is the interface that the specific implementation for the test jig being created needs to implement. The JigImplementation interface forces the specific implementation to add a run method, runPreProgramTests method and a runPostProgramTests. The run method is the main loop for the test sequence for the jig being programmed, and is the method that is called by the DetectorJig Class if it is found that the test jig stack corresponds with this jig implementation.

- **GenericTestBoard**

The GenericTestBoard class represents the a generic test jig board. It allows the

test jig system to invoke any of the actions discussed in section 3.6.3 (reading voltages, reading inputs etc). The class also adds a method to read the select line Id from the generic test jig board, which will allow the detector class to determine which test jig stack is connected to the two (or more) generic test jig board. When creating a new test jig implementation, the GenericTestBoard class is inherited from to create two specific implementations of test jig boards. In the example used throughout this project, the two boards created are McuTestBoard0 and McuTestBoard1. The 0 and 1 represent the 5 bit value of their select lines. The constructor for the McuTestBoard0 and McuTestBoard1 take in a com port string (this is the name of the comm port on the system that will be used to connect to the generic test jig board). Other than the constructor, the only other thing that is added to the McuTestBoard0 and McuTestBoard1 is a number of public constants that indicate what is connected to the inputs and outputs of the generic test jig board - in other words this is where all the logical mapping happens.

- **DetectorJig**

The DetectorJig is a descendant of a GenericTestJig. The GenericTestJig class represents a physical test jig, which has a test jig stack and a number of generic test boards, a label printer to print labels, etc. It gives the DetectorJig the ability to auto detect which com port is connected to which generic test board, as well as reading the 10 bit of the test jig stack. Once the DetectorJig exists, and is run (this method is called by the TestJigRunner object) the DetectorJig will assume it has a new jig attached to it. It will then read the number of com ports that are attached to the test jig in question from a configuration file (this will usually be 3, for 2 generic test jig boards and a UUT debug connection), and then go through the process of automatically detecting which com ports contain GenericTestJig boards and which are currently not communication (since at this point the UUT will not be powered). Once the com ports are detected, they will be placed in an array where the size of the array corresponds to the number of com ports, the final entry in the array is the UUT debug port, and the other entries in the array the GenericTestBoard com ports sorted in order of their 5 bit ids. From here the 10 bit test jig stack Id is read, and the specific implementation that corresponds to that id is instantiated.

The Generic Test Jig Framework fits on top of an existing code base that has been developed at Util Labs over the last few years. Part of this framework is the DeviceTestConnection communication class, which allows serial communication through

standard com ports. All messages moving through the DeviceTestConnection class will be logged to file on the test jig PC. This means at any time the results of a test, or any part of a test can be traced with a timestamp, allowing very precise tracking of when problems arrive which should allow for very accurate tracking of failure trends while using the generic test jig system.

3.8.1 Creating the TestBoards

Using as an example the implementation of the MCU test board that has been the guinea pig for this project all along, the following two classes were created:

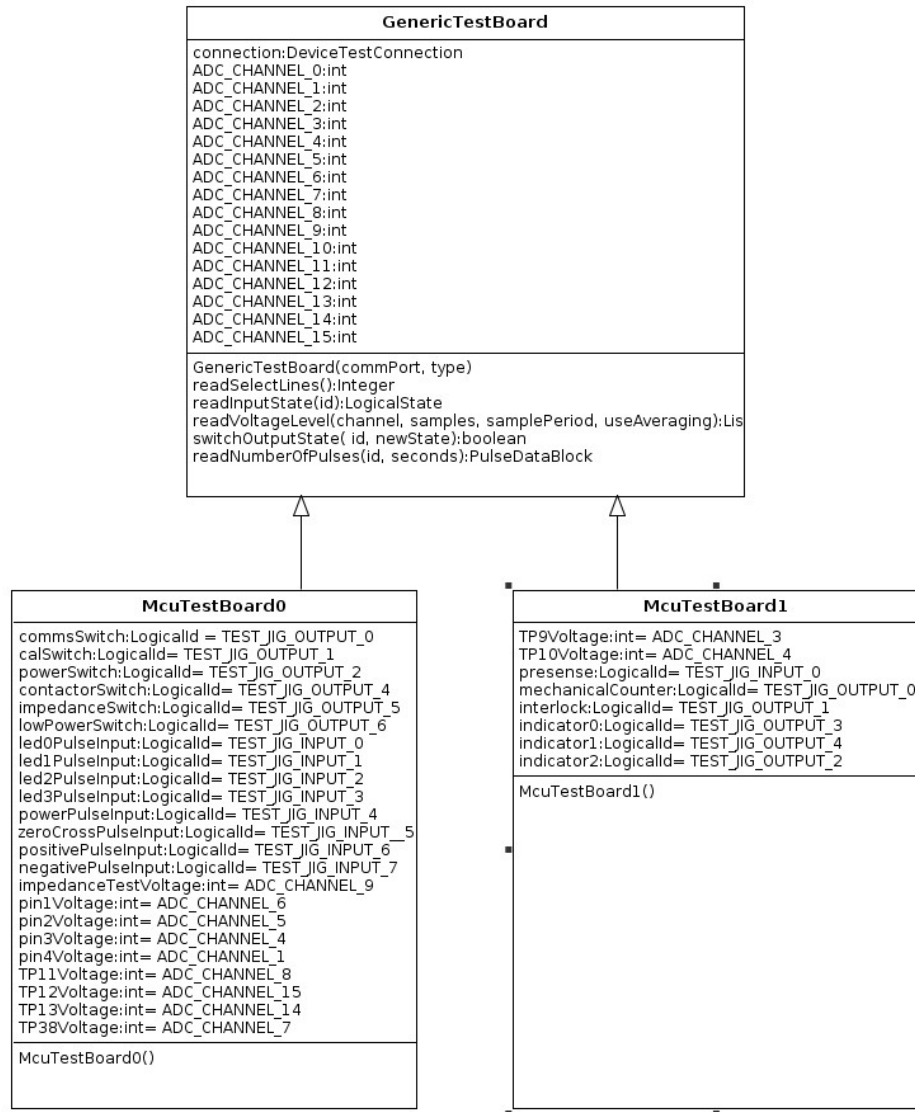


Illustration 55: TestBoard Specific Implementations

The two implementations both overwrite GenericTestBoard and just add a number of constants to it which allow the inputs and outputs on the generic test jig boards to be used

in the McuJigImplementation, and to ensure that the wrong input/output is not referenced on the wrong board (TEST_JIG_OUTPUT_1 exists on both of the McuTestBoard implementations, but those inputs are mapped to calSwitch and mechanicalCounter respectively). The McuJigImplementation will use these constants for all of the interactions with the generic test jig boards.

3.8.2 Creating the Implementation

Once the generic test boards have been mapped out, the actual test sequence code can be written. Once again the MCU test jig example is used.

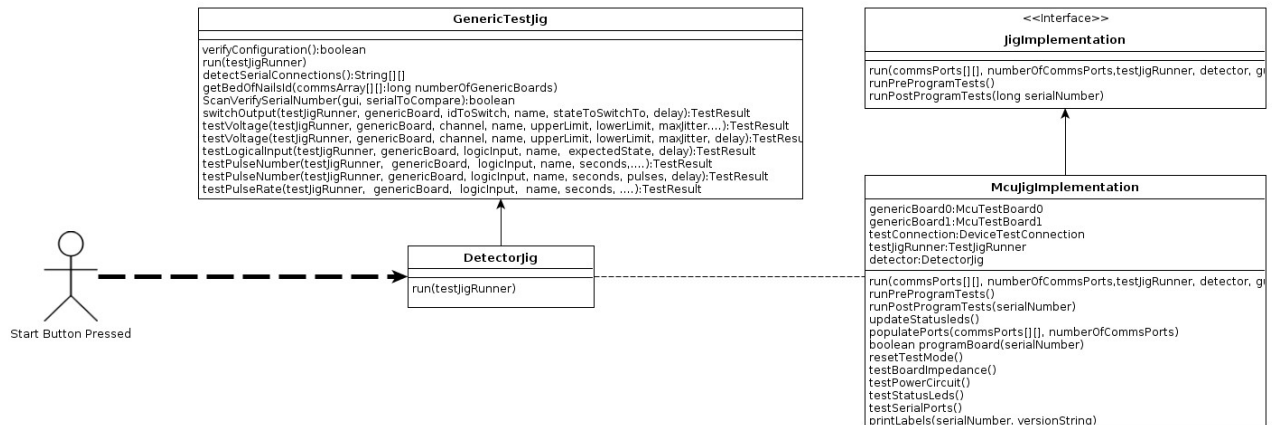


Illustration 56: Test Jig Implementation In The Framework

To add the implementation to the Java system, first the Implementation class needs to be created. This class implements the JigImplementation interface and as such inherits its run methods. The standard tests are already available to the Implementation through the Detector Jig (which is passed into the run method when invoked), but any tests that are not simple measurements and require some set up before the time need to be added to the implementation. As such the following tests are added to the implementation:

- resetTestMode
- testBoardImpedance
- testPowerCircuit
- testStatusLeds
- testSerialPorts

Each of these tests require some set up before they can be run (for example, before the board impedance impedance can be measured through the adc, the test jig stack must first set the low impedance relays to the correct state, and then switch on the low power supply before the measurement can happen). Generally these special tests have 3 phases nl. Set up, measure and tear down. The set up of the test happens when the appropriate preliminary steps are performed before a measurement can be taken (for example setting the relays to the right state in the impedance test example). The measurement happens once the tst has been set up, and the tear down happens once the measurement has already been taken, and the test jig is then returned to its original state.

When the run method of the Implementation is invoked, a TestJigRunner object is passed in. This is the object that all the results of tests should be reported to. The TestJigRunner can be queried to find out if any of the previous tests have failed. Generally in a test sequence, if any one test fails the entire jig fails. On failure, a failure code can be passed into the TestJigRunner as well, which will allow the printer to print the failure code which is then stuck to the board when the operator removes it from the test jig to facilitate failure tracking. The testJigRunner also allows the Implementation to write data to the history log of the test jig, so for example pass or failure conditions can be saved in the file, so that later when the file is analysed for historical data, it is both parseable to read the messages out of its history, but also human readable as the log information will be saved as plain text. The TestJigRunner also allows logging to the console to indicate the status of the tests to the operator.

There are 3 phases that most boards go through when being tested nl. The Pre-programming phase, the programming phase, and the post programming phase. The pre-programming phase handles all hardware on the UUT that required no interaction with programmable devices, for example power supplies, checking impedances, checking opto couplers etc. After the board has been tested that it is safe to start programming the programmable device(s) on it, the programming phase happens. If the device(s) is successfully programmed, then the post programming tests are run. Post programming tests are tests that require some interaction with the processor for them to pass, for example checking that an external memory chip is working by writing a file to it from the programmable device on the UUT and reading it out again. In order to get full test coverage it is important to ensure as much of the interaction on the UUT is tested. For example if a Flash memory chip is being tested before the programming has happened,

the chip may pass the test of reading and writing data, but if there is a bad solder joint on the pins on the programmable device that will normally uses that chip, by running the test through the programmable device, this error will be missed.

3.8.3 *Add The Implementation To The Detector*

Once the implantation for a test jig stack has been completed, this implementation can be added to the Detector run method.

```
long testJigId = getBedOfNailsId(commsPorts, (numberOfExpectedPorts-1));
TestJigid id = TestJigid.getEnum((int)testJigId);
JigImplementation implementation = null;
switch (id){
    case MCU_TEST_JIG:
        implementation = new McuJigImplementation();
        implementation.run(commsPorts, numberOfExpectedPorts, testJigRunner, this, gui);
        break;
    case INVALID_ID:
        break;
}
```

Illustration 57: Adding The Implementation To The Detector

The code snippet above shows where the new implementation needs to be inserted in the DetectorJig to allow it to be run when it's id is discovered. The test jig ids are stored in the TestJigid enumeration, and a new implementation with a new id needs to be added in there before code can be written for it.

3.10 CONCLUSION

The first successful fully integrated test using a test jig stack made using the generic test jig framework was run on Thursday 9th of June 2011. Previous tests were done but this one was the first one where the generic test jig framework created a board stack that fitted into a mechanical housing, and could be used by an operator to test a PCB (In this case the MCU V3 Rev D board), which passed all of the tests as expected.

The Iterative design approach proved to be very suitable to this project since even though there was an initial specification put forward by the internal customer (in this case the production department of Util Labs), this specification proved to be incomplete and it was only through a process of continual feedback and review that a suitable system was finally designed. Through the 5 iterations, all of the initial design requirements were met, and a number of features not initially anticipated to be necessary were added to create a framework that should greatly increase the test capabilities of Util Labs and its contract manufacturing facilities.

CHAPTER 4

4.1 INTRODUCTION

The generic test jig system over 5 iterations has become the de-facto standard method of testing hardware at Util Labs. The MCU V3 board that was used throughout this project as a guinea pig was chosen simply because it is the most complex board in the Util Labs product line to test. Since Iteration 4, 4 of the Util Labs products have so far been implemented in the generic test jig system (The Eddi display, The LCD front end module to the MCU V3, and the generic test board itself), and the decision has been made to backdate the tests to all existing devices in the product range as well. Design integration is also working very well because all 4 devices are still in development, and every time a new version of hardware is released the test jig associated with that hardware can be updated almost immediately (as can be seen by how quickly the changes to the to the guinea pig board were absorbed in Iteration 4 and 5).

In addition to being the de-facto standard at Util Labs, it has also attracted the attention of other players in industry, and there is an opportunity to commercialise the system by implementing the communication between the Test jig PC and the UUT in more generic way to allow other protocols to be implemented.

This chapter analyses the performance of the generic test jig system when measured against the requirements in chapter 1 and the product specifications set out in chapter 2.

4.2

RESULTS – USER REQUIREMENTS SPECIFICATION

Attribute	Requirement	Result
Cost	Physical cost per test fixture must be < R20 000	The Current Test Fixture comes to less than R20 000 (not including the label printer), and in addition to this the ability to share mechanical jigs between manufacturing runs for different products pushes the cost down even lower
Scalability	Must be able to add more test fixtures to a production run to increase production output at a cost of < R20 000 per test fixture	Since the test fixtures can share a printer on a network, the cost of adding another test fixture is just the cost of the pc and bed of nails stack if a set of spare mechanical fixtures is kept in stock so < R5000
Flexibility	Design must be generic and flexible enough to be able to design the hardware and software for a moderately sized jig in 1 month	The current system of templating allows for a jig that does not require any special tests or protocol changes to be designed in <1 week, without a programmable device on the UUT < 2 days.
Integration	Design should be integrated well enough with current design systems, that if a test pin needs to be moved or a test needs to be re-written that the impact will be less than 2 man days.	Between the 4 th and 5 th iteration, the guinea pig board changed, and since no additional tests were required (the test pins moved around and the leds on the board were removed), a jig could be made that was compatible with both the new and old version of the board in <2 days.

Attribute	Requirement	Result
Maintenance	Costs should be low enough that a hot swappable set of jigs can be kept on hand for failure conditions for under R20 000, and install time low enough that for line stoppages <30min on jig break down.	With the jig able to auto detect the test jig stack, in the event of a broken pin or electrical damage to the bed of nails, the lifting plate can be dropped and the test jig stack swapped in less than 20 mins with a cost of <R1000. Spare Generic test boards can also be kept on hand for < R300 a set
Traceability	The jig should support traceability in the factory, and remove some of the responsibilities from the operator to sort faulty boards into the correct areas.	By adding the ability to print failure modes on the same printer that prints serial numbers for manufactured units, there is now the ability to trace failures through the factory and easily make sure faulty boards arrive at the right station.

Table 11: Results - User Requirements Specifications

The generic test jig system conforms to all of the requirements set down by the original user requirements specification.

4.3 RESULTS – PRODUCT SPECIFICATION

Functional Specifications:

Attribute	Requirement	Result
Inputs	Jig must have at least 10 binary inputs, 100k Ω input impedance and be 20V tolerant on logic pins.	As a base with 2 test boards, 18 inputs are available and can be expanded. The input characteristics have been split away from the generic jig to the stack, and as such the impedance can be pushed into the M Ω range by buffering with op-amps, and the pins have been tested to 20V levels on the MCU test board with no problems.
Voltages	Jig must have at least 10 voltage inputs, 100k Ω input impedance and be 20V tolerant on adc pins with at least 1% accuracy.	As a base with 2 test boards, 18 inputs are available and can be expanded. Signal conditioning on stack can buffer the input impedance to M Ω levels. Pins tested with 20V input voltage successfully. The slowest conversion speed is used on the ADC to ensure the specified ± 2 bits accuracy on 12 bits is reached.
Pulse in	Jig must be able to measure at least 4 frequency inputs up to 2kHz, voltage tolerant to 20V	Using the external interrupts on the STM32F103 allows every input in the system to also be a pulse input with the same voltage characteristics. The external interrupts can sense pulses of as short as the width of the internal APB2 clock (36Mhz). Pins have been tested successfully with 1kHz signals

Attribute	Requirement	Result
Counters	Jig must be able to keep a count of pulses on at least 2 inputs, up to 2000 pulses per second, voltage tolerant to 20V	The same fast input module that handles the pulse in can also act as a counter with the same characteristics. The counter value is returned as a 32bit DWord value, which will then be the maximum counter value.
Comms	The test jig must have at least 2 serial communication lines to the unit under test @ Standard UART communication of at least 115200 baud, 8 bits, no parity, 1 stop bit	There is as a standard only 1 communication line to the UUT instead of 2, but a second can easily be added if required. The second communications port was meant to be a programming port and this was exchanged for a dedicated JTAG in the test jig.
Signal Injection	The test jig must be able to inject 5 or more signals up to 1kHz, at voltages up to 230Vac (for mains injection) up to 1A.	Outputs are capable of driving 500mA loads, and as such, D2n telecommunication relays were used on the guinea pig board which have a maximum voltage of 230Vac, can handle 3A loads and frequency up to 100Mhz. ^[31]

Table 12: Functional Specifications Results

Electrical Specifications:

Attribute	Requirement	Result
Supply	The test jig must run on 230Vac and require no external power supply	The Test jig is powered by 230Vac, and if the option is used for an external supply, this supply is external to the test board, but still inside the mechanical Jig.
Connector	Power must be supplied by a standard IEC power connector.	An IEC power connector is used with a built in fuse
Standby	The test jig, when not injecting	When not running tests, the test jig will

Attribute	Requirement	Result
	signals must draw less than 2W of power	draw just over 1.5W (The majority of which is due to the ICouplers.)
Isolation	Isolation between the UUT and all communications lines must be > 1kV	All lines are isolated with ICoupler devices that offer 2.5kV isolation
Fuse	The Test Jig must have a replaceable fuse	The fuse is built into the power socket, and is replaceable.
Physical Isolation	The operator must not be able to touch any of the test pins while the test is running.	There is a mechanical interlock that can be activated during the test to stop the operator opening the jig while the test is running.

Table 13: Electrical Specifications Results

Interface Specifications:

Attribute	Requirement	Result
Storage	Test result storage must be non-volatile and be able to store at least 10 000 results	Results stored to file, size is limited only to hard disk size
Access	Test results need to be accessible to a windows or Linux operating system	Test Jig Application is native to Linux
Interface	Once Installed, the required user interaction should consist of no more than single actions, for example: close lid, push button, scan label etc	As Specified
Comms	RS232/RS485 communication will take place at 115200 baud, 8 data bits, no parity bit, 1 stop bit	The ICouplers and USB-Serial converters both handle at least 115200 baud
Language	Graphical communication to the user must all be in English	As Specified

Table 14: Interface Specifications

4.3 CONCLUSION

As can be seen from the comparison tables, the generic test jig system has conformed to all of the requirements placed at the start of this project.

CHAPTER 5

5.1 INTRODUCTION

The project up until this point has been a resounding success as can be seen by the results obtained in the previous chapter. Throughout the course of the project, a number of ideas have come about that would make the generic test jig framework significantly more powerful, not only as a design tool, but as a commercial product. There were however also some things that could have been done better, and it is with this in mind that the project is reviewed.

5.2 CONCLUSIONS

The Iterative design approach was definitely the best approach to take in this project, as the initial requirements were not all available at the start of the project, and some things had to be added throughout the course of the design. However, since each iteration of the design required changes to be made to the design, a large number of PCBs were made throughout the course of the project. In the context of Util Labs this was not a problem as there is an agreement in place with our preferred PCB manufacturer that tooling costs are not added to the manufacturing cost of prototype board due to the large number of boards we order, but even taking this into account, the project went over budget by about 20%. Iterative design is a very powerful tool, but in a predominantly hardware design project, more time should be spent in the first iteration designing a proper simulation environment (like for example the cardboard mock-up system used in iteration 2 could be used more as it is a cheap and simple way to see if everything fits together properly).

An interesting theme throughout working on this project and interacting with different stakeholders, is that technology as a concept in a design is not confined to the technical quality of the solution. The best ideas to come out of this project have been around Templating, and the ability to cut down on design time, rather than the architectural design of the code or even design of the actual hardware. In future, I think it would be a good idea to spend a little more time getting the stakeholders in the project more involved, specifically the non-technical stakeholders as unique technologies can grow from such interactions.

An interesting side note on the project though is the commercial potential that has arisen. At the start of the project during the literature review, a number of systems were explored, and the two systems that are closest in function to the generic test jig system are flying probe systems and traditional beds of nails. During a normal design cycle, products are designed, and tested by hand, but flying probe systems offer an alternative to this technique, as they can quickly be programmed to become a virtual test fixture for a prototype. However when the product is then completed, a bed of nails permanent test fixture can be designed which then allows the product to go into mass manufacture. The templating process has cut the design time so low on the generic test jig system that there is an opportunity to compete with flying probe systems, as for a simple board, a test fixture can be completed within a week (provided the PCBs arrive from the manufacturer in time),

which puts it in serious competition with these systems. The competitive edge however, is that a flying probe system has no way to migrate from a “prototype” tester to a “production” tester, which is where the generic test jig system fits in. This has been discussed internally, and a few initial discussions have been made with some people in industry on the concept, and the initial feedback is very positive. This will definitely be something that will be explored in future.

5.3 RECOMMENDATIONS

The following recommendations are made by the author to ensure the most is got out of the project from this point forward:

- The generic test jig system is already being established as the de-facto standard for testing at Util Labs, but it would be useful to backdate this process to all of the other products in the Util Labs portfolio. The ability to use the same operators on all test equipment and to rapidly change the production line to a different kind of product is a valuable commercial “edge”.
- The templating process has made the design of test jigs simple enough, that with some training, other departments in Util Labs can take advantage of the technology – specifically the quality department that consistently needs to test and measure on products that come out of the field presenting some unknown problem. A number of jigs could definitely streamline the process of fault finding for known problems.
- Some additional tests are required for upcoming products in the Util Labs portfolio – specifically capacitive sense buttons need to be tested. This functionality should be added soon.
- The commercial value of the generic test jig system in its own right should not be ignored. There are no products currently on the market that can offer the same level of production test capability within the time frame offered by this system!

5.4 PROPOSED FURTHER WORK

As is the nature of iterative design, there are always a new directions to look into to make a product better. This is a list of useful functionality that the author feels would be very valuable to continue developing:

- Camera interface
It would be useful to have a simple camera interface to do automated visual inspection of LCD screens on products.
- Standardised network bridge
The current debug information is sent to and from the UUT via RS232, but there is no reason that a small processor cannot be placed as an intermediary between the UUT and the test jig pc which will then allow the conversion of messages from RS232 to UART, SPI, I2C etc etc.
- Programmer support
There are only so many widely used programming interfaces in industry today, and it would be useful if the generic test jig natively supported SWIM and serial bootloading of the major processors from ST, ATMEL microchip etc.
- Port To ARM11/ARM9
The current system of having the test application on the test jig PC next to the jig works well, but a better solution would be to run Linux on a small ARM11/ARM9 mini computer board, and build the board directly into the test jig so that it is a single unit.
- Web Server
A web server should be installed on the test jig PCs to allow for easy retrieval of logs as well as simple updates of the Java application in situations where not test jig implementations are added.
- Reporting
Create Cron jobs on the system to email a list of people a daily report of the number of devices test on the test jig, and a breakdown of different failures that were found to allow for easy tracking of failure trends.
- Automated templates
The templating process can be fully automated, and any time cut off of the design will mean faster delivery of test equipment.